Bharat Ratna Indira Gandhi College of Engineering, Solapur

Master of Computer Application

SUBJECT:

FULL STACK DEVELOPMENT (MCAC301)

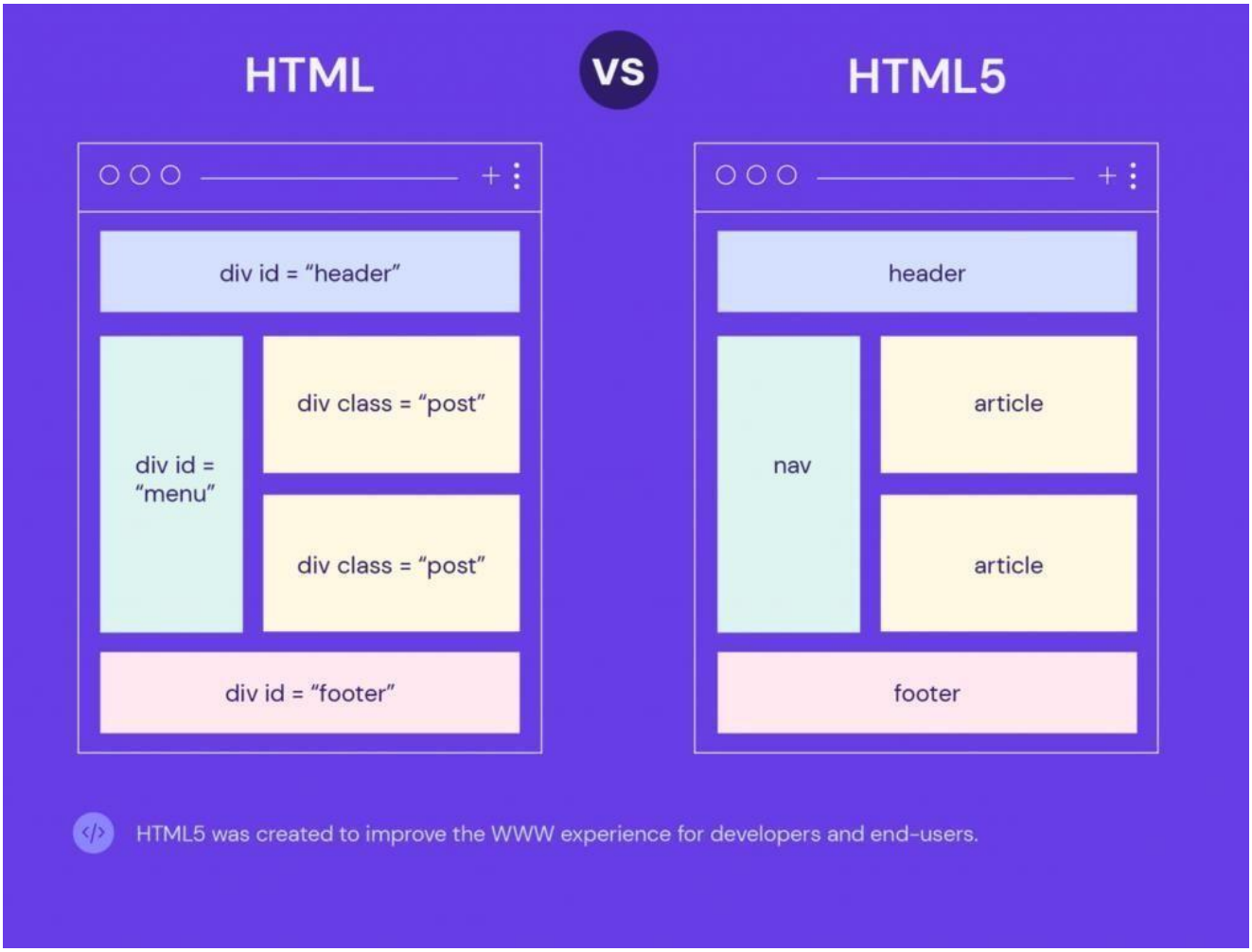
SEMESTER-III

UNIT 1: INTRODUCTION TO FRONT-END DEVELOPMENT

HTML

What is HTML?

- **HTML (HyperText Markup Language)**: The standard markup language used to create web pages.
- **Purpose**: To structure web content and create the skeleton of a web page.
- **Elements**: Consists of a series of elements, represented by tags, which define the structure and content of the web page.



HTML vs. HTML5

HTML

- **HTML 4.01**: The previous major version of HTML.
- **Features**:
- Based on SGML (Standard Generalized Markup Language).
- Uses elements such as `<font>`, `<center>`, `<strike>`, etc., for styling which are now deprecated.

HTML5

- **HTML5**:
  The latest version of HTML, introduced with the aim to improve support for multimedia and new graphic elements.

  - **Key Features**:
    - **New Elements**: Semantic elements like `<article>`, `<section>`, `<nav>`, `<header>`, `<footer>`.
    - **Enhanced Multimedia Support**: Built-in support for audio (`<audio>`) and video (`<video>`) elements.
    - **APIs and Graphics**: Canvas API for 2D graphics (`<canvas>`), support for scalable vector graphics (SVG), Geolocation API, Web Storage, Web Workers, etc.
    - **Form Enhancements**: New input types (e.g., `email`, `url`, `date`, `range`), attributes (e.g., `placeholder`, `required`, `autofocus`).

HTML Basic Tags with Description

Document Structure Tags

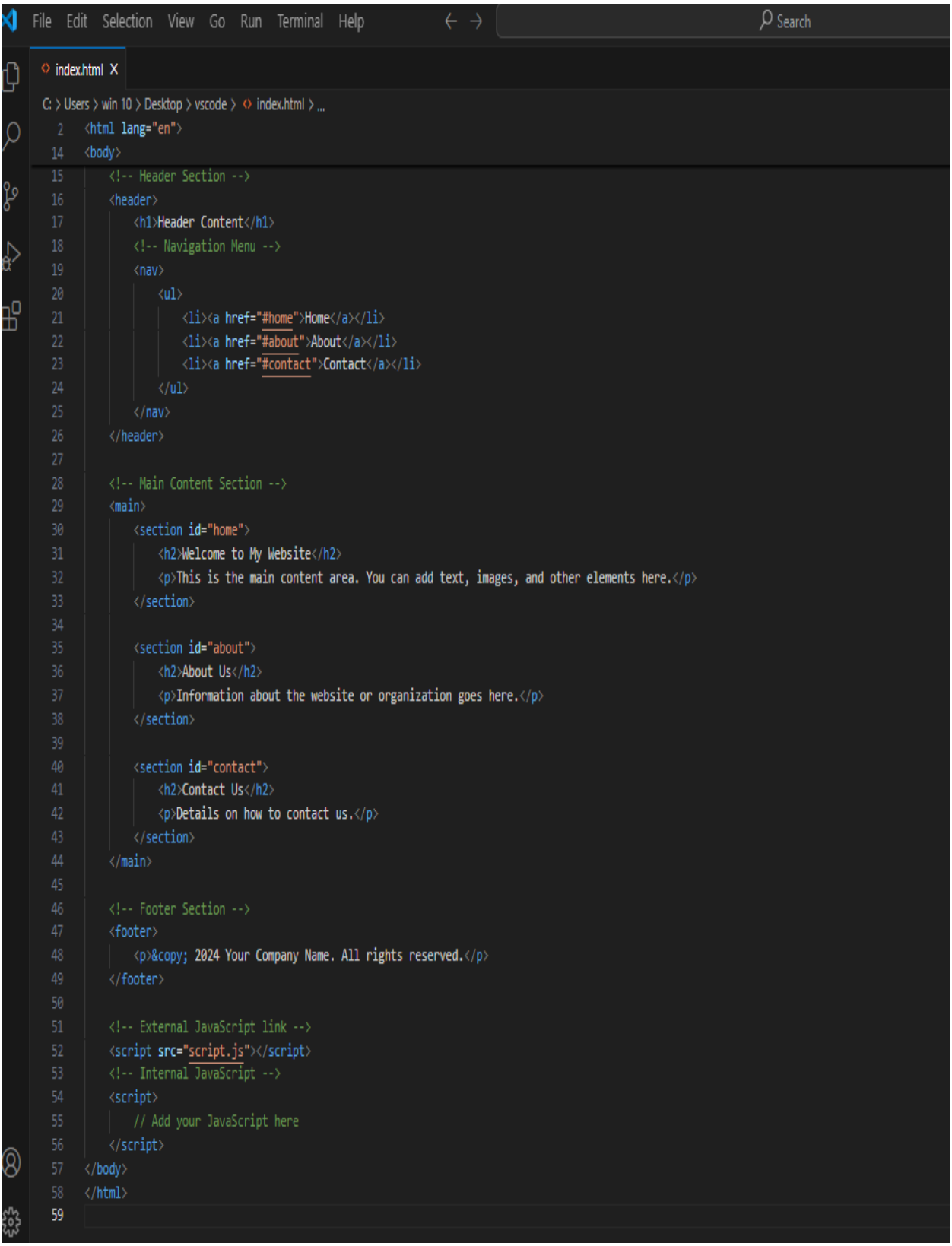
- `<!DOCTYPE html>`: Declares the document type and version of HTML.
- `<html>`: The root element of an HTML document.
- `<head>`: Contains meta-information about the document (e.g., title, meta tags, links to stylesheets).
- `<body>`: Contains the content of the HTML document.

Metadata Tags

- `<title>`: Sets the title of the document, shown in the browser's title bar or tab.
- `<meta>`: Provides metadata about the HTML document (e.g., character set, author, description, keywords).

Content Sectioning Tags

- `<header>`: Defines the header section of a document or section.
- `<nav>`: Defines a navigation section with links.
- `<section>`: Defines a section in a document.
- `<article>`: Defines an independent, self-contained article.
- `<aside>`: Defines content aside from the main content (e.g., sidebar).
- `<footer>`: Defines the footer section of a document or section




```html
<html lang="en">
<body>
    <!-- Header Section -->
    <header>
        <h1>Header Content</h1>
        <!-- Navigation Menu -->
        <nav>
            <ul>
                <li><a href="#home">Home</a></li>
                <li><a href="#about">About</a></li>
                <li><a href="#contact">Contact</a></li>
            </ul>
        </nav>
    </header>

    <!-- Main Content Section -->
    <main>
        <section id="home">
            <h2>Welcome to My Website</h2>
            <p>This is the main content area. You can add text, images, and other elements here.</p>
        </section>

        <section id="about">
            <h2>About Us</h2>
            <p>Information about the website or organization goes here.</p>
        </section>

        <section id="contact">
            <h2>Contact Us</h2>
            <p>Details on how to contact us.</p>
        </section>
    </main>

    <!-- Footer Section -->
    <footer>
        <p>&copy; 2024 Your Company Name. All rights reserved.</p>
    </footer>

    <!-- External JavaScript link -->
    <script src="script.js"></script>
    <!-- Internal JavaScript -->
    <script>
        // Add your JavaScript here
    </script>
</body>
</html>
```

❖ Text Content Tags

- **<h1> to <h6>**: Define HTML headings, <h1> is the highest and <h6> is the lowest.
- **<p>**: Defines a paragraph.
- **<a>**: Defines a hyperlink.
- **
**: Inserts a line break.
- **<hr>**: Defines a thematic change in the content, typically a horizontal rule.

List Tags

- **<ul>**: Defines an unordered (bulleted) list.
- **<ol>**: Defines an ordered (numbered) list.
- **<li>**: Defines a list item.

➢ Media Tags

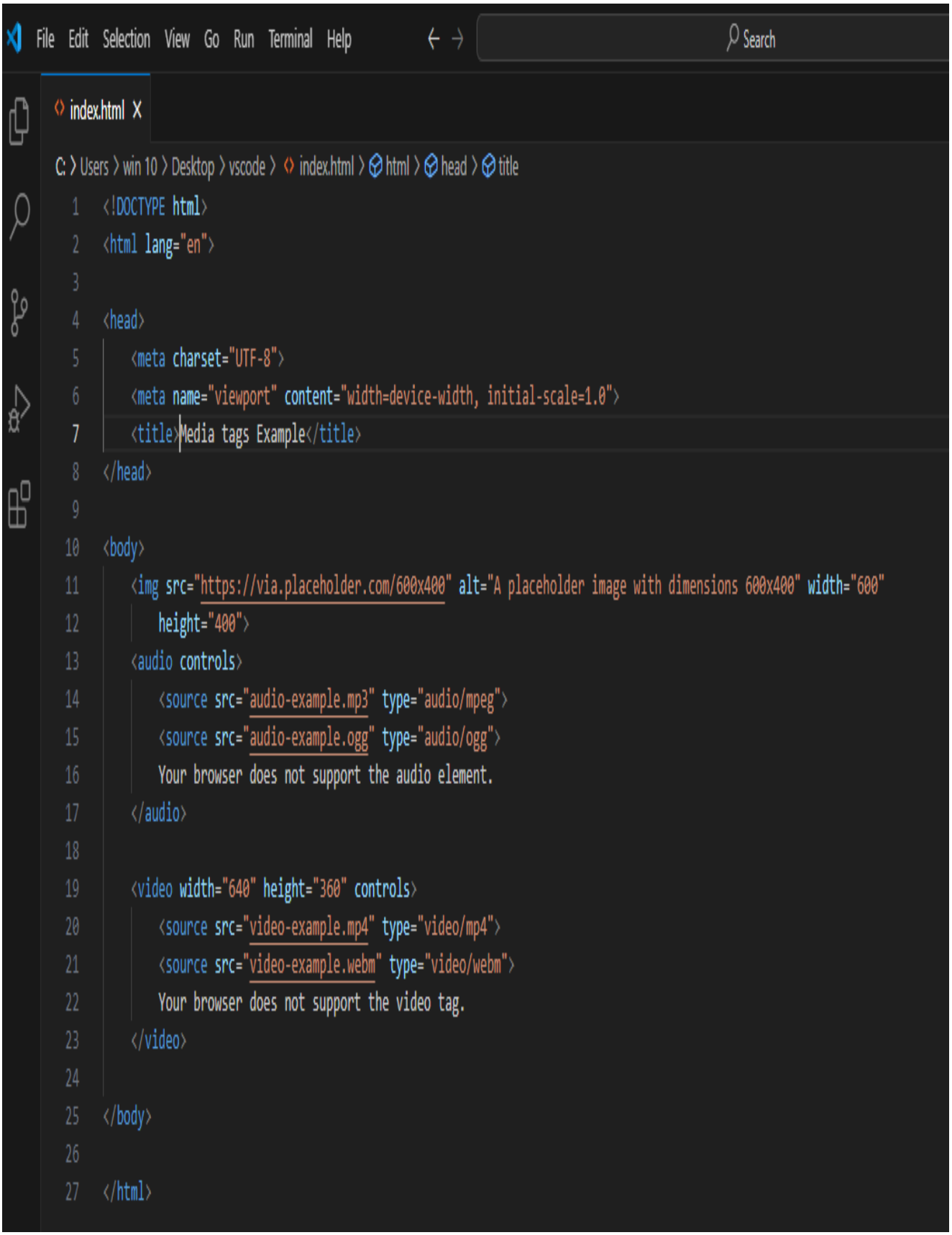
- <img>**:** Embeds an image.

Common attributes for <audio> & <video>

- **src**: Specifies the URL of the image file.
- **alt**: Provides alternative text for the image if it cannot be displayed. This is also important for accessibility and SEO.
- **width & height**: Set the width and height of the image in pixels. These attributes are optional but help to control the image's size.

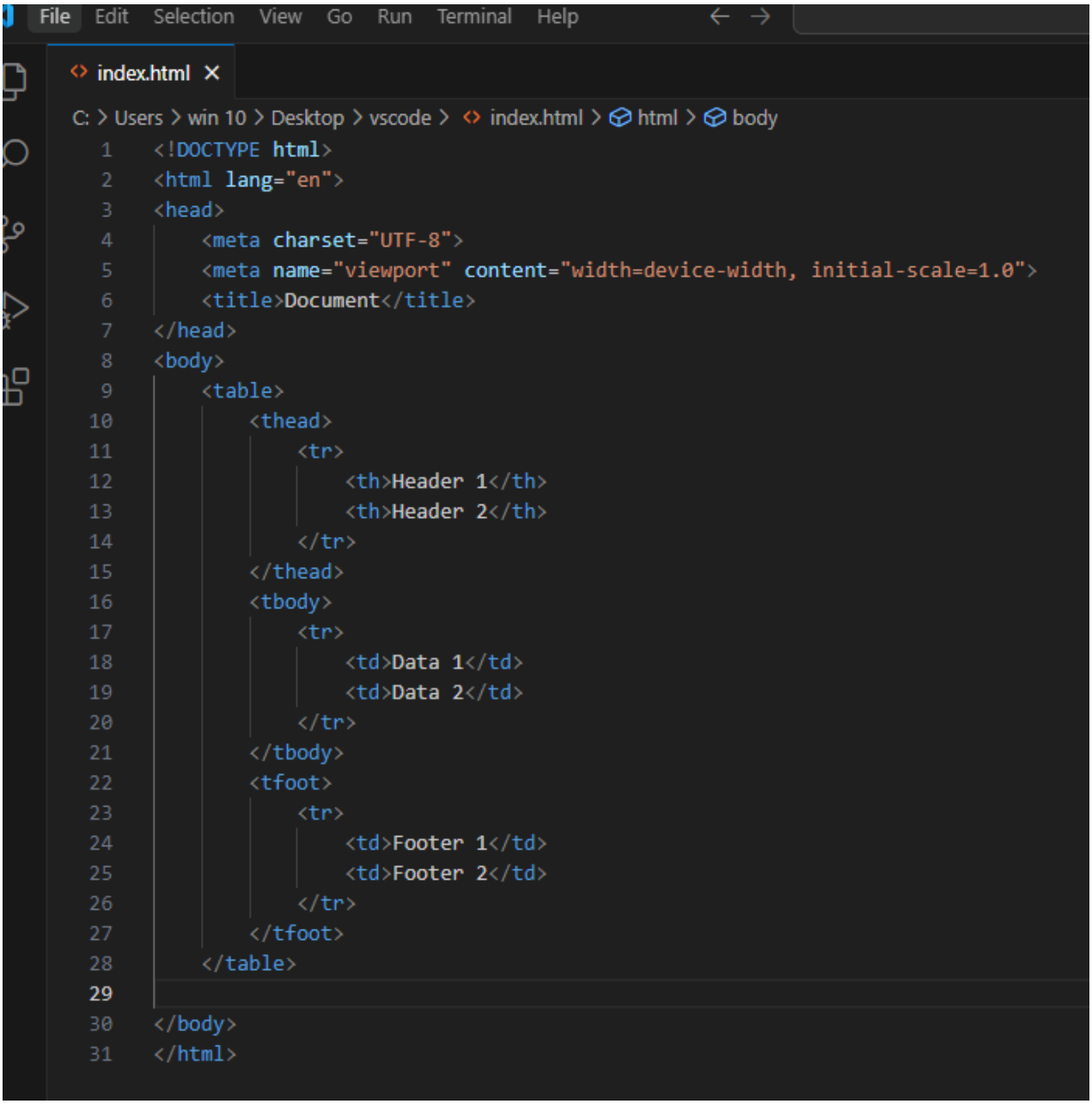
- **<audio>**: Embeds audio content.
- **<video>**: Embeds video content.


File Edit Selection View Go Run Terminal Help
Search
index.html X
C: > Users > win 10 > Desktop > vscode > index.html > html > body > section > h3
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>HTML Boilerplate with Common Tags</title>
</head>
<body>
<header>
<h1>Main Header (H1)</h1>
</header>
<section>
<h2>Subheader (H2)</h2>
This is a paragraph. It contains some text to demonstrate the <p> tag.
<h3>Sub-subheader (H3)</h3>
Another paragraph here, to show how <p> tags are used. You can also use the anchor tag to create lin
<h4>Smaller Header (H4)</h4>
More text in a paragraph. Add a link here to navigate to another page or section.
<h5>Even Smaller Header (H5)</h5>
Yet another paragraph. Use
 to insert a line break. For example:
First line.
Second line.
<h6>Smallest Header (H6)</h6>
Here's a horizontal rule below:
<hr>
Following the horizontal rule. This separates content visually.
</section>
<footer>
© 2024 Your Company. All rights reserved.
</footer>
</body>
</html>


File Edit Selection View Go Run Terminal Help
Search
index.html X
C: > Users > win 10 > Desktop > vscode > index.html > html > head > title
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Media tags Example</title>
</head>
<body>
<img src="https://via.placeholder.com/600x400" alt="A placeholder image with dimensions 600x400" width="600"
height="400">
<audio controls>
<source src="audio-example.mp3" type="audio/mpeg">
<source src="audio-example.ogg" type="audio/ogg">
Your browser does not support the audio element.
</audio>
<video width="640" height="360" controls>
<source src="video-example.mp4" type="video/mp4">
<source src="video-example.webm" type="video/webm">
Your browser does not support the video tag.
</video>
</body>
</html>

➢ Table Tags

- `<table>`: Defines a table.
- `<tr>`: Defines a table row.
- `<th>`: Defines a header cell in a table.
- `<td>`: Defines a standard cell in a table.



➢ Form Tags

- `<form>`: Defines an HTML form for user input.
- `<input>`: Defines an input control.
- `<label>`: Defines a label for an input element.
- `<textarea>`: Defines a multi-line text input control.

Example Based on form tags :

1. **<form>**: Defines the start of the form. The action attribute specifies where the form data should be sent when the form is submitted, and the method attribute defines the HTTP method to be used (usually "get" or "post").
2. **<label>**: Defines a label for an input element. The for attribute should match the id of the corresponding input.
3. **<input>**: Various types of input elements are used here:
    - type="text" for a single-line text input.
    - type="email" for email input, which validates the email format.
    - type="password" for password input, which hides the entered characters.
4. **<fieldset>** and **<legend>**: Group related elements (like radio buttons) and provide a title for that group.
5. **<select>**: Creates a dropdown list. The name attribute is used to identify the form data when it is submitted.
6. **<option>**: Defines options in a dropdown list. Each <option> tag specifies an option within the <select> tag.
7. **<button>**: Defines a button to submit the form.

index.html

C: > Users > win 10 > Desktop > vscode > index.html > html > body > form

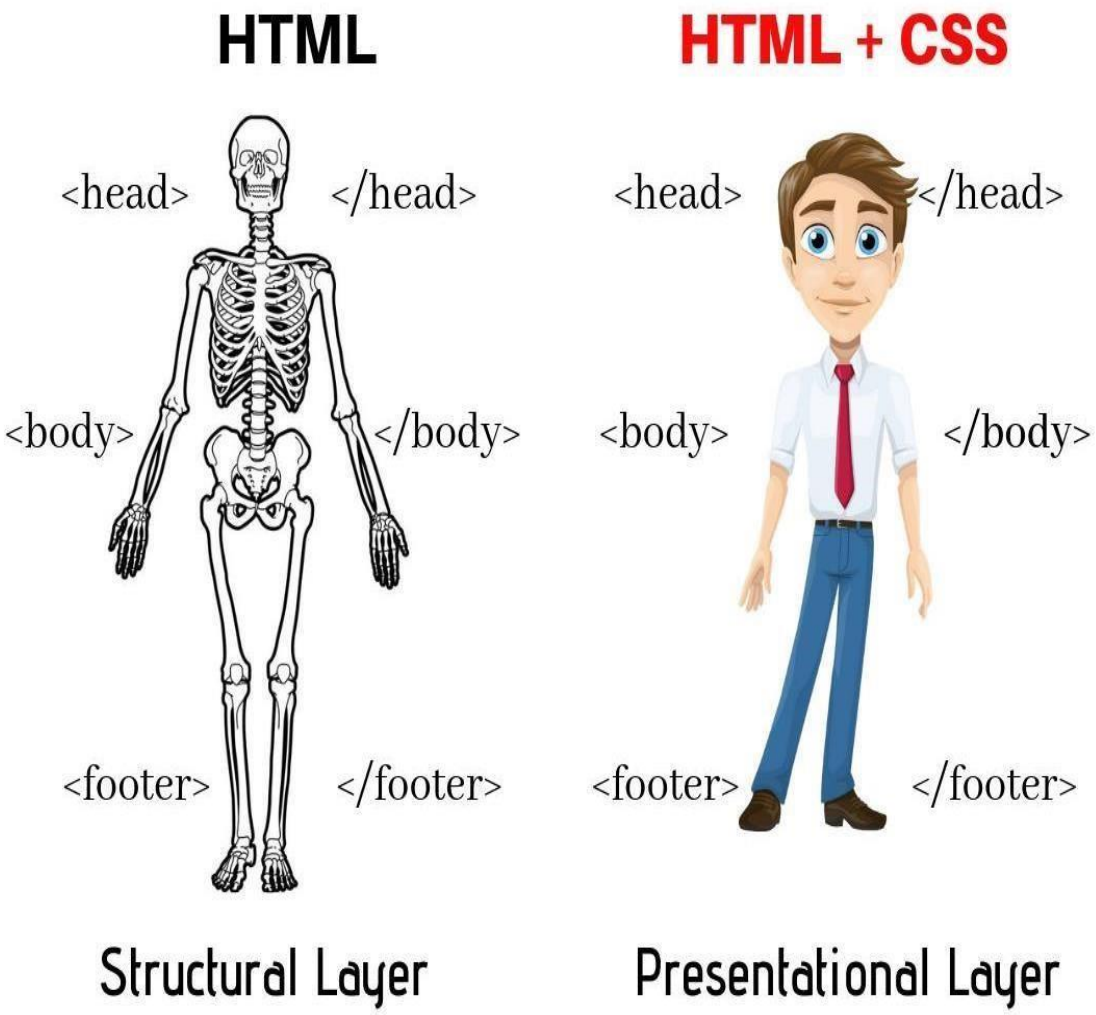
```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>HTML Form Example</title>
</head>
<body>
    <h1>Sample Form</h1>
    <form action="/submit" method="post">
        <!-- Text Input -->
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>
        <!-- Email Input -->
        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>
        <!-- Password Input -->
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required>
        <!-- Radio Buttons -->
        <fieldset>
            <legend>Gender:</legend>
            <label>
                <input type="radio" name="gender" value="male" required> Male
            </label>
            <label>
                <input type="radio" name="gender" value="female" required> Female
            </label>
            <label>
                <input type="radio" name="gender" value="other" required> Other
            </label>
        </fieldset>
        <!-- Select Dropdown -->
        <label for="country">Country:</label>
        <select id="country" name="country" required>
            <option value="">Select a country</option>
            <option value="us">United States</option>
            <option value="ca">Canada</option>
            <option value="uk">United Kingdom</option>
            <option value="au">Australia</option>
        </select>
        <!-- Submit Button -->
        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

Semantic HTML5 Tags

- **<main>**: Specifies the main content of a document.
- **<figure>**: Specifies self-contained content, like illustrations, diagrams, photos.
- **<figcaption>**: Defines a caption for a `<figure>` element.
- **<mark>**: Highlights text.
- **<progress>**: Displays a progress bar.

---***---

HTML Vs CSS


HTML
HTML + CSS
<head>
</head>
<body>
</body>
<footer>
</footer>
<head>
</head>
<body>
</body>
<footer>
</footer>
Structural Layer
Presentational Layer

CSS

➢ What is CSS ?

- **CSS (Cascading Style Sheets)**: A stylesheet language used to describe the presentation of a document written in HTML or XML.
- **Purpose**: To separate content (HTML) from design (CSS) and control the layout, colors, fonts, and overall look of web pages.

Basic CSS Syntax

```
css  selector {
  property: value;
}
```

- **Selector**: Targets the HTML elements to be styled.
- **Property**: Specifies the aspect of the element to be styled (e.g., color, font-size).
- **Value**: Defines the setting for the property (e.g., red, 16px).

Example:

```
body
{
background-color:
   lightblue;
}
h1 {
color: navy;
fontsize:24px
    }
```

➢ CSS vs CSS3

CSS

- **CSS1**: Introduced in 1996, the first version of CSS.
- **CSS2**: Released in 1998, added more properties and better control over layout.

  Features: Basic styling capabilities, layout control, fonts, colors, and positioning.

CSS3

- **CSS3**: The latest version, modularized into different sections (modules) for easier updates and maintenance.
- **Key Features**:
  - **Selectors**: Attribute selectors, pseudo-classes, and pseudo-elements.
  - **Box Model Enhancements**: Border-radius for rounded corners, box-shadow for shadows.
  - **Backgrounds and Borders**: Multiple backgrounds, gradients, border images.
  - **Text Effects**: Text shadow, custom fonts with @font-face
  - **2D/3D Transformations**: Translate, rotate, scale, and skew elements
  - **Transitions and Animations**: Smooth transitions and keyframe animations.
  - **Flexbox and Grid Layouts**: Flexible and grid-based layouts for responsive design.

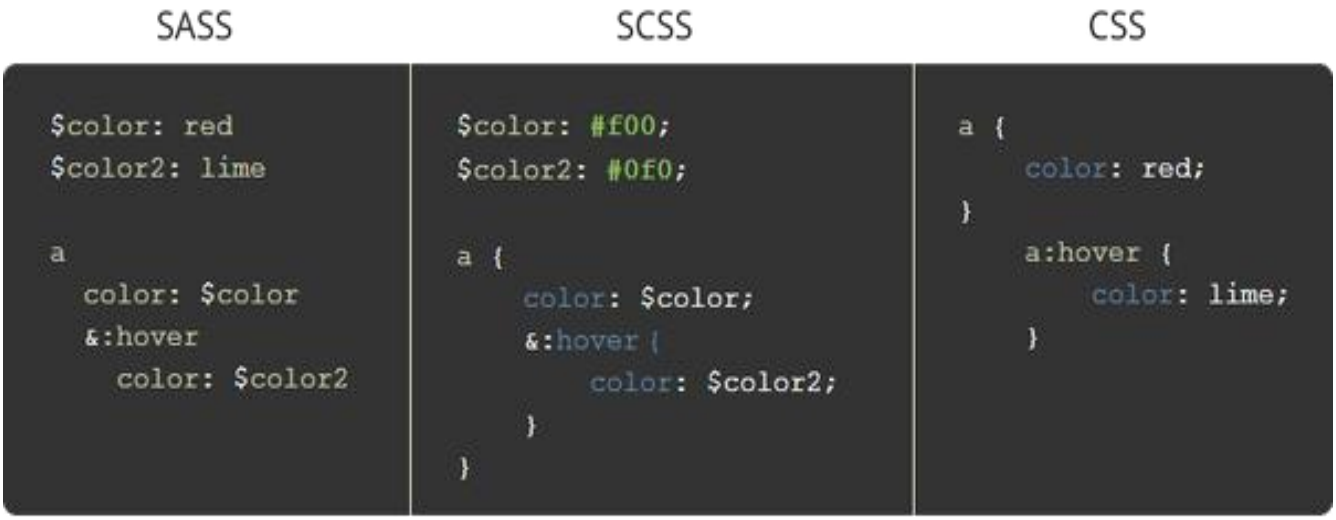
CSS3 vs CSS

Comparison Chart

CSS	CSS3
CSS is the basic version with the basic formatting functionality.	CSS3 is the latest iteration of the CSS language that extends the functionality of CSS2.
It does not support responsive design and cannot handle media queries.	It supports responsive design and can handle media queries quite well.
CSS is relatively slower than CSS3.	CSS3 is the latest evolution which is much faster than its previous iterations.
It does not support 3D transformations and animations.	You can create 3D transformations, transitions, and animations using CSS3.
CSS cannot be split into varied modules.	CSS3 can be split into modules.
It has old and standard colors.	It offers exciting new ways to play with colors.

DB Difference Between.net

CSS vs CSS3 vs SCSS vs Sass



CSS vs CSS3

- **CSS**: The foundation, providing basic styling and layout control.
- **CSS3**: Extends CSS with advanced features for better design, animations, and responsive layouts.

SCSS (Sassy CSS)

- **SCSS**: A syntax of Sass, fully compatible with the syntax of CSS3.
- **Features**: Variables, nesting, mixins, inheritance, partials, and more.

  Syntax Example:

```
scss
$primary-color: #333;
body {      color:
$primary-color;
header
{
```

```
background-color: lighten($primary-color,
20%);
} }
```

Sass (Syntactically Awesome Style Sheets)

- **Sass**: A preprocessor scripting language that is interpreted or compiled into CSS.
- **Features**: Similar to SCSS, but uses an indentation-based syntax.

  Syntax Example:

```
$primary-color:
#333  body
color: $primary-
color
   header background-color: lighten($primary-
color, 20%)
```

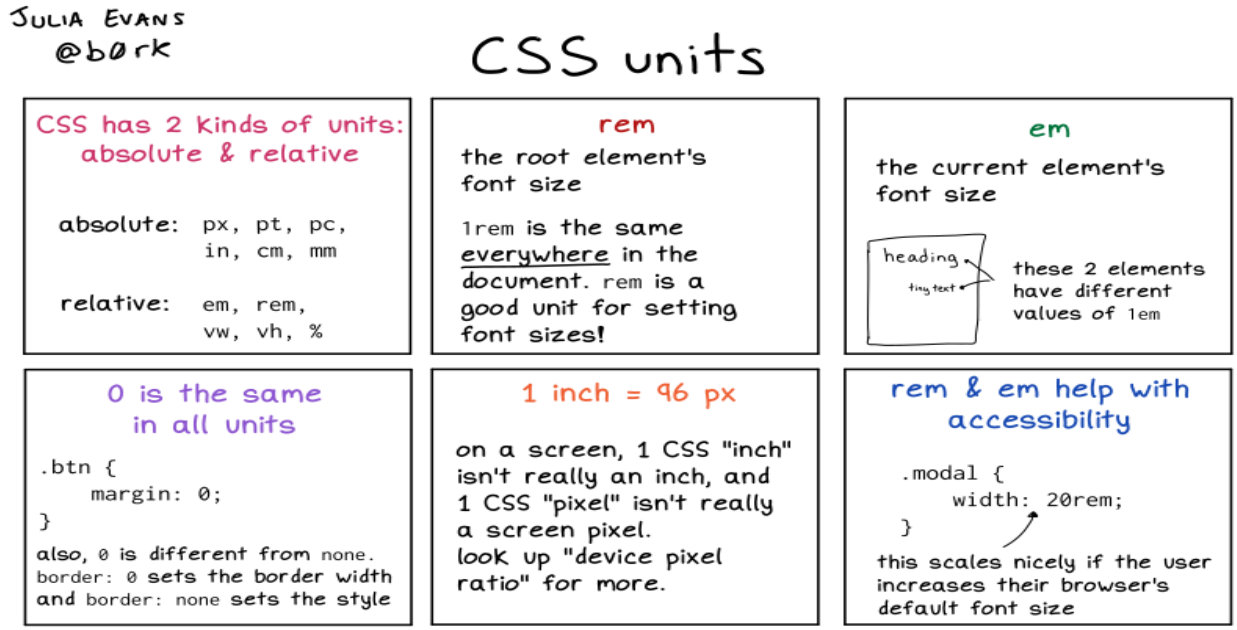
➢ CSS Units

➢ Absolute Units

- **Pixels (px)**: Fixed unit relative to the display.
- **Inches (in)**, **Centimeters (cm)**, **Millimeters (mm)**: Physical units.
- **Points (pt)**, **Picas (pc)**: Units used in print media.

➢ Relative Units

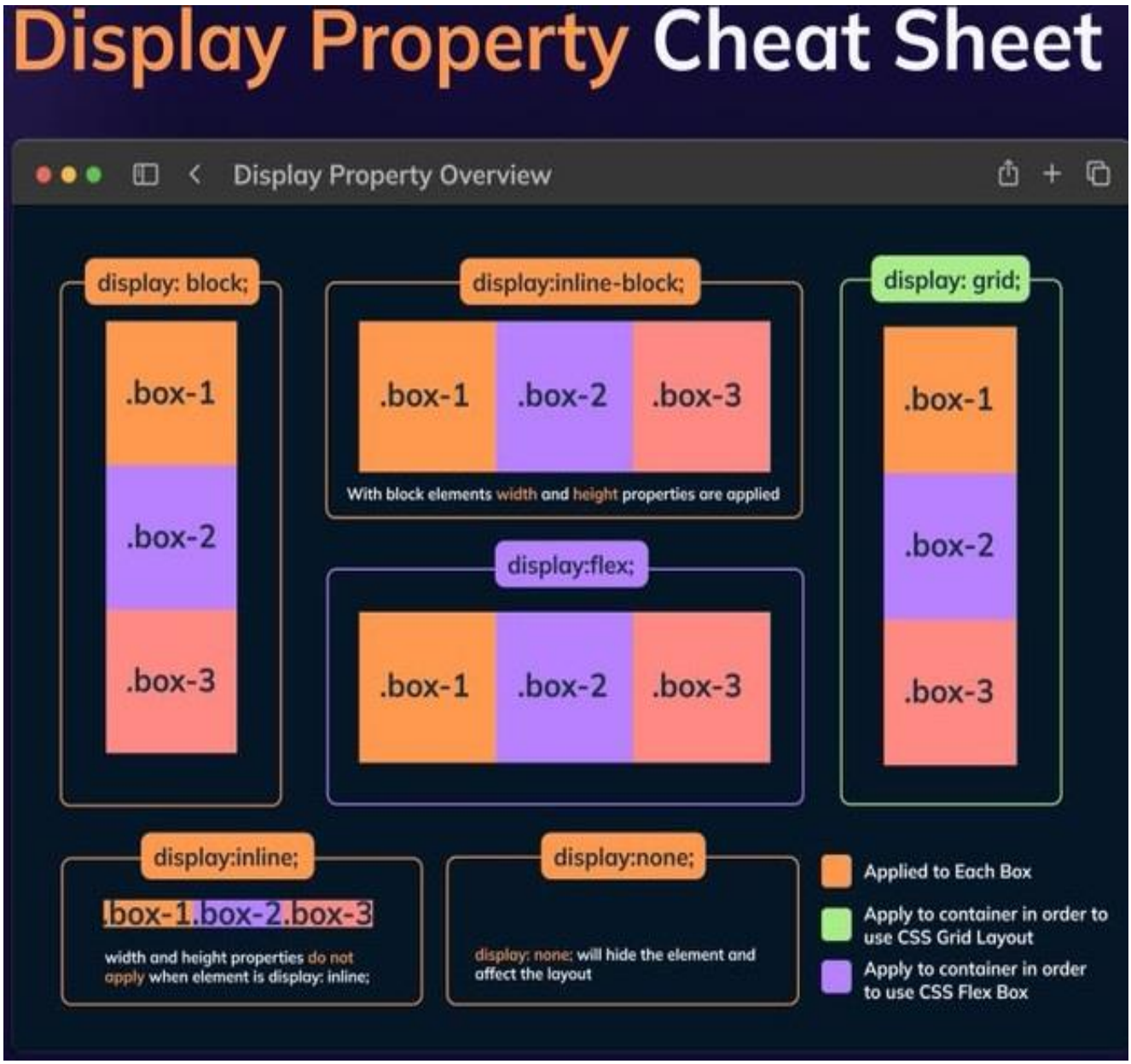
- **Percentages (%)**: Relative to the parent element.
- **Ems (em)**: Relative to the font-size of the element.
- **Rems (rem)**: Relative to the font-size of the root element.
- **Viewport Width (vw)**, **Viewport Height (vh)**: Relative to 1% of the viewport's width and height.
- **Viewport Min (vmin)**, **Viewport Max (vmax)**: Relative to the smaller or larger of the viewport's width and height.



➢ CSS Basic Properties

➢ Display

- **display**: Specifies the display behavior of an element.
- **block**: Element takes up the full width.
- **inline**: Element takes up only as much width as necessary.
- **inline-block**: Allows setting width and height, but remains inline.
- **none**: Element is not displayed.



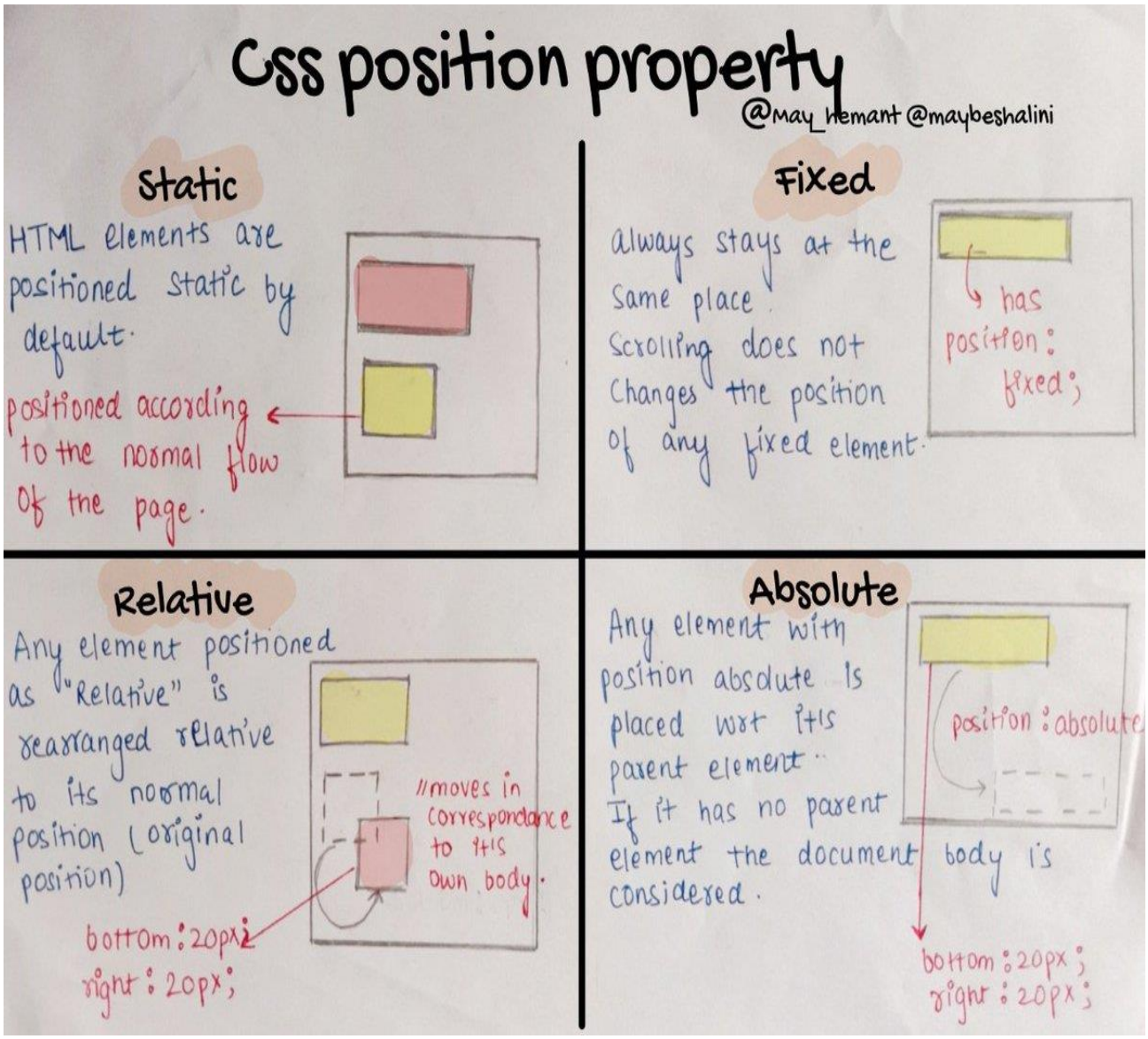
Example:

```
.block-element {
display: block;
}
.inline-element {
display: inline; }
```

➢ Position

Description: Specifies the positioning method for an element.

- **static**: Default positioning
- **relative**: Positioned relative to its normal position.
- **absolute**: Positioned relative to its nearest positioned ancestor.
- **fixed**: Positioned relative to the viewport.
- **sticky**: Toggles between relative and fixed, based on the scroll position.



Example:

```
.relative-element {
position: relative;
top: 10px;
left: 20px;
}

.absolute-element {
position: absolute;
top: 50px;
left: 50px;
 }
```

- **Box-Sizing**

- **box-sizing**: Defines how the width and height of an element are calculated.
- **content-box**: Width and height include only the content.
- **border-box**: Width and height include padding and border.

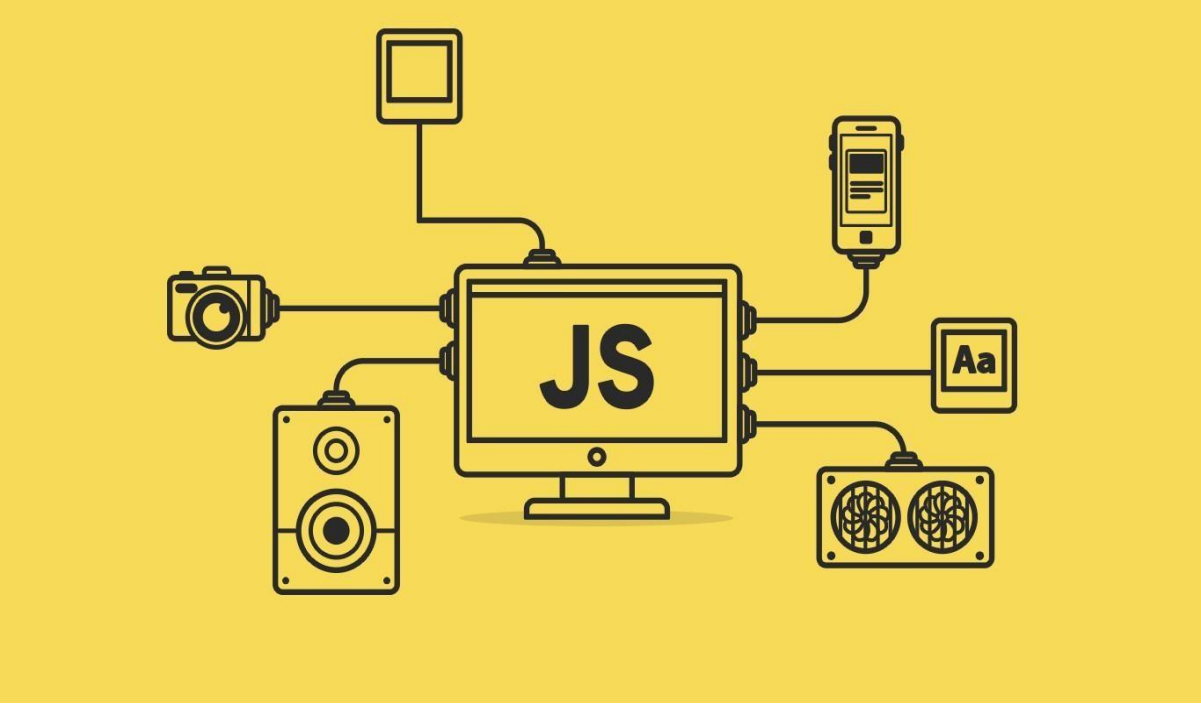
Example:

```
 .content-box-element {
box-sizing: content-box;
    width: 200px;
 padding: 20px;
 border: 10px solid
    black;
}
```

```
.border-box-element {
boxsizing: border-box;
width: 200px;
padding: 20px;
border: 10px solid black;
}
```

___***___

JavaScript



What is JavaScript?

- **JavaScript**: A high-level, interpreted programming language primarily used for web development.
- **Purpose**: To create interactive and dynamic web pages by manipulating the Document Object Model (DOM), handling events, and communicating with servers.

Why Use JavaScript?

- **Interactivity**: Allows for dynamic content updates, form validations, animations, and event handling.

- **Versatility**: Can be used on both the client-side (in the browser) and server-side (using environments like Node.js).
- **Popularity**: Widely supported by all modern web browsers, with a vast ecosystem of libraries and frameworks (e.g., React, Angular, Vue.js).

➢ JavaScript Fundamentals

- **Variables**: Containers for storing data values.

```
var name ="Alice";

let age = 25;
const pi = 3.14;
```

- **Data Types**: Includes numbers, strings, booleans, objects, arrays, functions, null, and undefined.

```
let number = 42;
let text = "Hello,World!"; let
numbers = [1,2,3,4,5];
let isTrue = true;
let person = {name: "John", age: 30};
```

- **Operators**: Arithmetic, comparison, logical, and assignment operators.

```
let sum = 10 + 5; // Arithmetic
let isEqual = (10 == "10"); // Comparison

let isBothTrue = (true && false); // Logical

let x = 10; x += 5; // Assignment
```

- **Control Structures**: Conditional statements and loops.

```
// Conditional statements
 if (age > 18) {
 console.log("Adult");
} else {
    console.log("Minor");
}

 // Loops
 for (let i = 0; i<5; i++) {
console.log(i);
}  let j = 0;
while (j < 5) {
console.log(j);     j++;
}
```

- **Functions**: Blocks of code designed to perform a specific task.

```
function greet(name){
return `hello ${name}`;
}

Let greeting=greet(“Alice”);

Console.log(greeting);//O/p=>Hello,Alice
```

➢ Basic Required Features of JavaScript

- **DOM Manipulation**: Changing the content and style of HTML elements.

```
document.getElementById("myElement").innerHTML = "New
Content"; document.querySelector(".myClass").style.color =
"red";
```

- **Event Handling**: Adding interactivity to web pages.

```
document.getElementById("myButton").addEventListener("click"
, function()
{
alert("Button
clicked!");
});
```

- **AJAX**: Asynchronous JavaScript and XML for server communication without reloading the page.

```
let xhr = new XMLHttpRequest();
xhr.open("GET", "data.json", true);
xhr.onload = function() {
 if (xhr.status === 200) {
console.log(xhr.responseText);
 }
}; xhr.send();
```

➢ ES6 Features

- **let and const**: Block-scoped variables.

```
let name = "Alice";

const pi = 3.14;
```

- **Arrow Functions**: Shorter syntax for writing functions.

```
const add = (a, b) => a + b;
```

- **Template Literals**: String literals allowing embedded expressions.

```
let name = "Alice";

let greeting = `Hello, ${name}! `;
```

- **Destructuring**: Unpacking values from arrays or properties from objects.

```
let [a, b] = [1, 2];
let {name, age} = {name: "Alice", age: 25};
```

- **Default Parameters**: Setting default values for function parameters.

```
function greet (name = "Guest")
{
    return `Hello, ${name}!`;
}
```

- **Rest and Spread Operators**: Handling multiple elements.

```
function sum(...numbers) {
   return numbers.reduce((total, num) => total + num, 0);
}

let arr1 = [1, 2, 3];

let arr2 = [...arr1, 4, 5,6];
```

- **Classes**: Syntax for creating objects and dealing with inheritance.

```
class Person {
constructor(name, age) {
this.name = name;
this.age = age;  }
greet() {
        return `Hello, my name is ${this.name}. `;
    }
}
```

- **Promises**: Handling asynchronous operations.

```
let promise = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Success"), 1000);
});

 promise.then(result => console.log(result));
```

➢ Asynchronous Programming

Callbacks

- **Function passed as an argument**: Called after a task is completed.

```
function fetchData(callback) {
setTimeout(() => {
callback("Data received");
},
1000);
}  fetchData((data) =>
console.log(data));
```

Promises

- **Object representing eventual completion or failure**: of an asynchronous operation.

```
let promise = new Promise((resolve, reject) => {
setTimeout(() => resolve("Success"), 1000);
});
 promise.then(result => console.log(result)).catch(error =>
console.log(error));
```

async/await

- **Syntax for writing asynchronous code**: in a synchronous manner.

```
async function fetchData() {
let response = await fetch("data.json");
```

```
let data = await response.json();
 console.log(data);
}
fetchData()
```

TypeScript

What is TypeScript?

- **TypeScript**: A typed superset of JavaScript developed by Microsoft.
- **Purpose**: To add static types to JavaScript, making it easier to identify and fix errors during development.

Why Use TypeScript?

- **Type Safety**: Detects errors and potential bugs at compile time.
- **Enhanced IDE Support**: Improved code completion, navigation, and refactoring.
- **Modern JavaScript Features**: Supports ES6+ features and compiles to plain JavaScript.
- **Scalability**: Better suited for large-scale applications with complex codebases.

TypeScript Features

- **Static Typing**: Define variable types explicitly.

```
let age: number = 25; let name: string = "Alice";
```

- **Interfaces**: Define custom types.

```
interface Person {
name: string;
 age: number;
}
 let person: Person ={
name: "John",
 age: 30
};
```

- **Classes and Inheritance**: Similar to ES6, with added type safety.

```
class Animal {
name: string;
constructor(name
: string) {
this.name =
name;      }
    move(distance: number) {
        console.log(`${this.name} moved ${distance}
meters.`);
    }
}
class Dog extends Animal {
bark() {
        console.log("Woof! Woof!");
   }
}
let dog = new Dog("Rex");
dog.bark();
dog.move(10);
```

- **Generics**: Create reusable components with type parameters.

```
function identity<T>(arg: T): T {
return arg;
}  \
Let output=identify<string>("hello");
```

Example:

```
Function
greet(person:string):string{ return
`Hello ,${person}`;
}
```

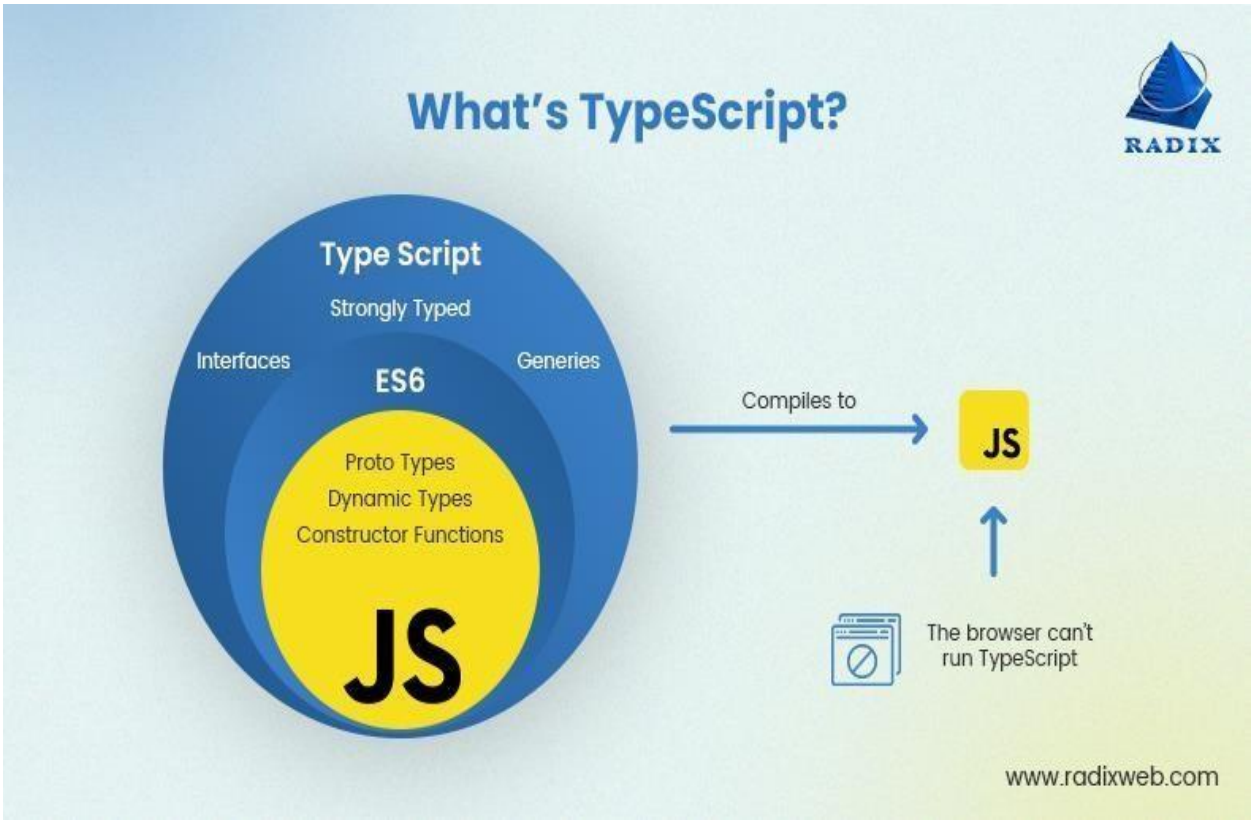
let user = 'John";

console.log(greet(user));

//outpout:Hello John

Compiling TypeScript

- **Compilation**: TypeScript code must be compiled into JavaScript using the TypeScript compiler (`tsc`). `tsc script.ts`



Angular

Unit 1: Introduction to Angular

What is Angular?

- **Angular**: A platform and framework for building single-page client applications using HTML, CSS, and JavaScript/TypeScript.
- **Developed by**: Google.
- **Version**: Angular (commonly refers to versions 2 and above; AngularJS is version 1.x).

Key Features

- **Component-Based Architecture**: Applications are built using components, which encapsulate the HTML, CSS, and logic for a part of the UI.
- **Two-Way Data Binding**: Automatic synchronization between the model and the view.
- **Dependency Injection**: A design pattern that allows a class to request dependencies from external sources rather than creating them.
- **Directives**: Extend HTML with custom elements and attributes.
- **Services and Dependency Injection**: Used to share data and logic across components.
- **Routing**: Built-in module for navigation between views.

Unit 2: Setting Up the Development Environment

Prerequisites

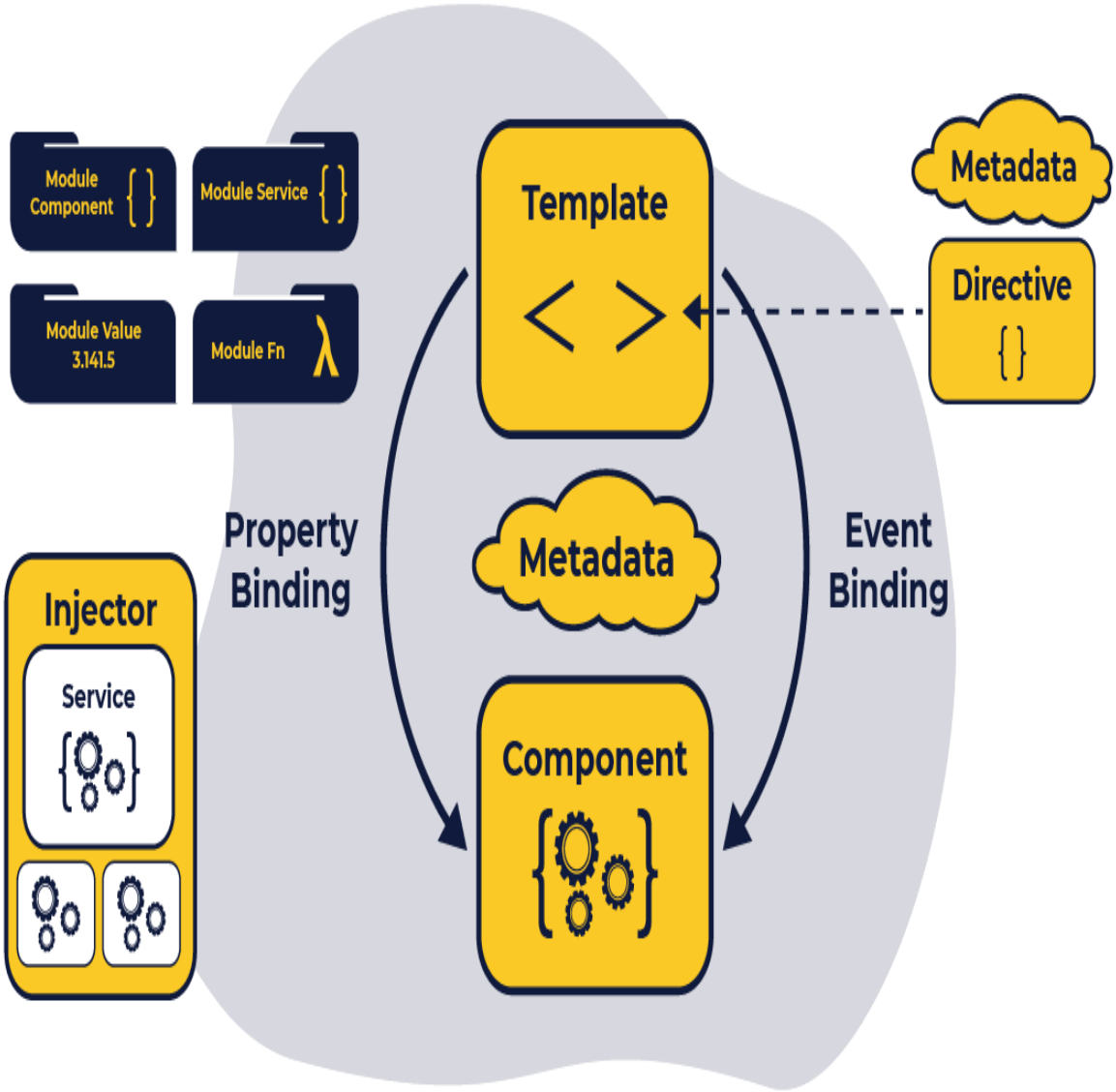
- **Node.js and npm**: Ensure Node.js and npm are installed.
- **Angular CLI**: Command-line interface tool for initializing, developing, scaffolding, and maintaining Angular applications.

Installation

```
npm install -g
@angular/cli
```

Creating a New Angular Project

```
ng new
projectname
cd projectname
ng serve
```


Module Component
Module Service
Module Value 3.141.5
Module Fn
Injector
Service
Template
Metadata
Component
Property Binding
Event Binding
Metadata
Directive

Folder Structure Of Angular Application :

```
app/                    --> all of the source files for the application
  app.css               --> default stylesheet
  components/           --> all app specific modules
    version/              --> version related components
      version.js                --> version module declaration and basic "version" value servi
      version_test.js           --> "version" value service tests
      version-directive.js      --> custom directive that returns the current app version
      version-directive_test.js  --> version directive tests
      interpolate-filter.js     --> custom interpolation filter
      interpolate-filter_test.js --> interpolate filter tests
  view1/                --> the view1 view template and logic
    view1.html            --> the partial template
    view1.js              --> the controller logic
    view1_test.js         --> tests of the controller
  view2/                --> the view2 view template and logic
    view2.html            --> the partial template
    view2.js              --> the controller logic
    view2_test.js         --> tests of the controller
  app.js                --> main application module
  index.html            --> app layout file (the main html template file of the app)
  index-async.html      --> just like index.html, but loads js files asynchronously
karma.conf.js         --> config file for running unit tests with Karma
e2e-tests/            --> end-to-end tests
  protractor-conf.js    --> Protractor config file
  scenarios.js          --> end-to-end scenarios to be run by Protractor
```

Unit 3: Angular Components and Templates

Components

- **Component**: A fundamental building block of an Angular application.
- **Creation**: Using Angular CLI.

```
ng generate component component-name
```

Component Anatomy

- **Decorator**: `@Component` decorator specifies the metadata.

```
@Component({
selector: 'app-component-name',
templateUrl: './component-
name.component.html',
 styleUrls: ['./component-name.component.css']
})
```

- **Template**: Defines the view for the component (HTML).
- **Styles**: Component-specific CSS.
- **Class**: Contains the logic and data for the component.

Data Binding

- **Interpolation**: Binding data from the component to the view.

```
<p>{{ title }}</p>
```

- **Property Binding**: Binding properties of HTML elements to data in the component.

```
<input [value]="title">
```

- **Event Binding**: Binding events to methods in the component.

```
<button (click)="onClick()">Click Me</button>
```

- **Two-Way Binding**: Combining property and event binding for form elements.

```
<input [(ngModel)]="title">
```

Unit 4: Directives and Pipes

Directives

- **Attribute Directives**: Change the appearance or behavior of an element.
  Example: ngClass, ngStyle.
- **Structural Directives**: Change the DOM layout by adding or removing elements.

  Example: ngIf, ngFor.

```
<div *ngIf="isVisible">Visible Content</div>
<ul>
  <li *ngFor="let item of items">{{ item }}</li>
</ul>
```

Pipes

- **Purpose**: Transform output in the template.
- **Built-in Pipes**: DatePipe, UpperCasePipe, LowerCasePipe, CurrencyPipe, etc.

```
<p>{{ today | date }}</p>

<p>{{ price | currency }}</p>
```

- **Custom Pipes**: Creating custom pipes.

```
ng generate pipe pipe-name
```

Unit 5: Services and Dependency Injection

Services

- **Purpose**: Share data and logic across components.
- **Creation**: Using Angular CLI.

  ng generate service service-name

Dependency Injection

- **Injecting Services**: Using constructor injection in components.

  constructor(private serviceName: ServiceName) {}

Unit 6: Routing and Navigation

Router Module

- **Setting Up Routes**: Define routes in the AppRoutingModule.

  const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },];

Router Outlet

- **Placeholder for Routed Components**: Defined in the main template.

  <router-outlet></router-outlet>

Navigation

- **RouterLink**: Directive for navigating between routes.

  <a [routerLink]="['/about']">About

Unit 7: Forms in Angular

Template-Driven Forms

- **FormsModule**: Import in AppModule

  import { FormsModule } from '@angular/forms';

- **Syntax**: Binding form controls to model properties.

```
<form #form="ngForm" (ngSubmit)="onSubmit(form)">
  <input name="name" ngModel>
  <button type="submit">Submit</button>
</form>
```

Reactive Forms

- **ReactiveFormsModule**: Import in AppModule.

  import { ReactiveFormsModule } from '@angular/forms';

- **FormBuilder**: Service to create form controls.

```
import { FormBuilder, FormGroup } from '@angular/forms';   constructor(private fb: FormBuilder) {
this.form = this.fb.group({
 name: [''],
 email: ['']
  });
}
```

Unit 8: HTTP Client Module

HTTP Requests

- **HttpClientModule**: Import in AppModule. import { HttpClientModule } from '@angular/common/http';
- **HttpClient**: Service for making HTTP requests.

```
import { HttpClient } from '@angular/common/http';

constructor(private http: HttpClient) {}  getData()
{   this.http.get('https://api.example.com/data')
   .subscribe(data => {      console.log(data);
   });
}
```

Unit 9: Angular Modules

NgModules

- **Purpose**: Organize an application into cohesive blocks of functionality.
- **Core Module**: The root module that bootstraps the application.

```
@NgModule({
declarations: [AppComponent, OtherComponent],    imports:
[BrowserModule, AppRoutingModule],
providers: [],
 bootstrap: [AppComponent]
})

export class AppModule { }
```

- **Feature Modules**: Modules for specific features or sections of the application.

```
@NgModule({
declarations: [FeatureComponent],
imports: [CommonModule],
exports: [FeatureComponent]
}) export class FeatureModule
{ }
```

Unit 10: Advanced Topics

Angular Animations

- **Animations Module**: Import in AppModule. import { BrowserAnimationsModule } from '@angular/platformbrowser/animations';

Testing

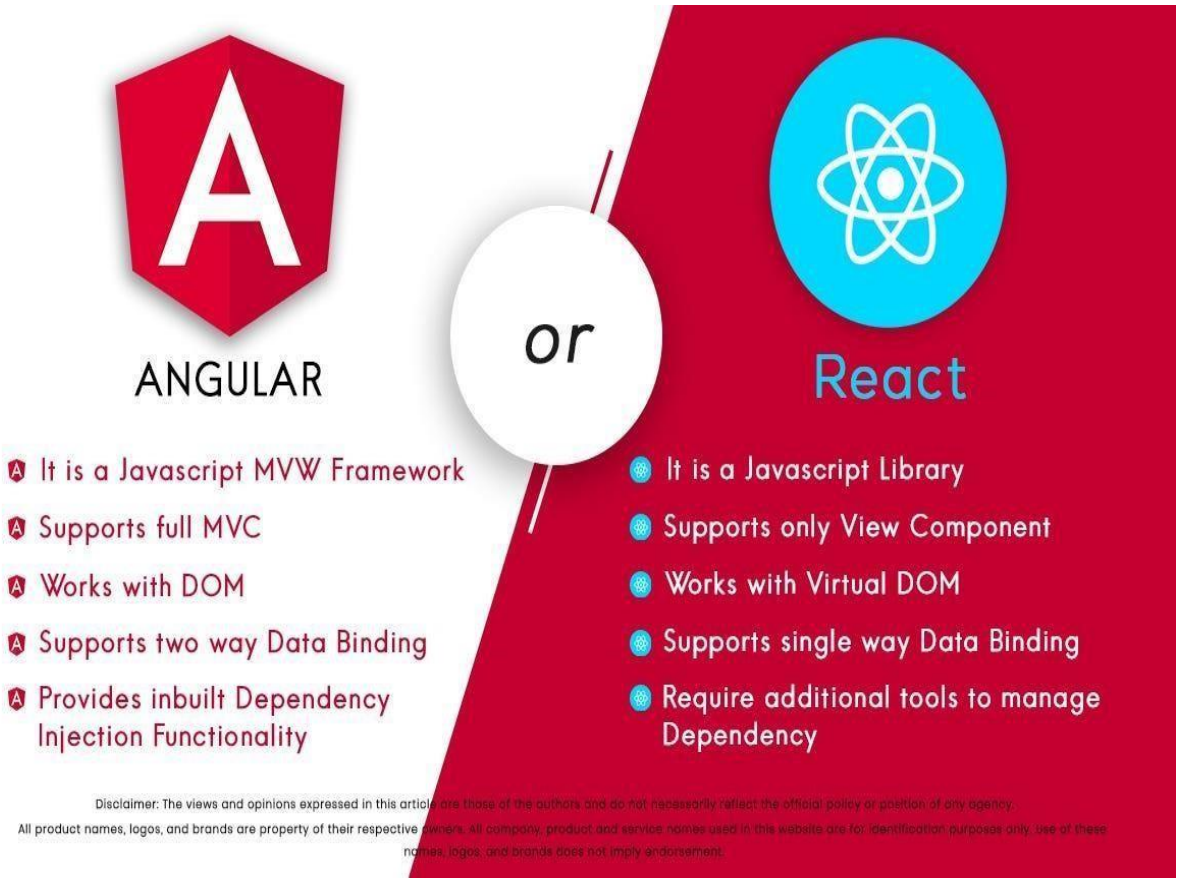
- **Unit Testing**: Using Jasmine and Karma.
- **End-to-End Testing**: Using Protractor.

Performance Optimization

- **Lazy Loading**: Load modules on demand.

```
const routes: Routes = [
 {
   path: 'feature',
   loadChildren: () =>  import('./feature/feature.module')
.then(m => m.FeatureModule) }
];
```

- **Ahead-of-Time (AOT) Compilation**: Compiles the application during the build process.
- **Server-Side Rendering (SSR)**: Rendering Angular applications on the server.



Angular vs React

Overview

Angular

- **Developed by**: Google
- **Initial Release**: 2010 (as AngularJS), 2016 (as Angular)
- **Type**: Full-fledged framework for building web applications
- **Language**: TypeScript
- **Architecture**: Component-based, follows MVC (Model-View-Controller) pattern

React

- **Developed by**: Facebook
- **Initial Release**: 2013
- **Type**: Library for building user interfaces
- **Language**: JavaScript (with JSX syntax extension)

1. Learning Curve

- **Angular**: Steeper learning curve due to its comprehensive nature, involving concepts like dependency injection, RxJS for reactive programming, and Angular-specific syntax and tools.
- **React**: Relatively easier to learn, especially for developers familiar with JavaScript. Focuses primarily on the view layer, which simplifies understanding.

2. Language and Syntax

- **Angular**: Uses TypeScript, a statically typed superset of JavaScript, which enhances code quality and maintainability.
- **React**: Uses JavaScript with JSX, a syntax extension that allows HTML-like code within JavaScript. React can also be used with TypeScript.

3. Data Binding

- **Angular**: Two-way data binding, which automatically synchronizes data between the model and the view.
- **React**: One-way data binding, which makes data flow more predictable and easier to debug.

4. Performance

- **Angular**: Performance can be optimized using features like change detection and Ahead-of-Time (AOT) compilation.
- **React**: Generally faster due to its virtual DOM implementation, which minimizes direct DOM manipulations and updates.

5. State Management

- **Angular**: Uses services and RxJS for state management. The NgRx library provides a more complex and robust solution for state management.
- **React**: Uses built-in hooks like **`useState`** and **`useReducer`** for state management. External libraries like Redux or MobX can be used for more complex state management needs.

6. Ecosystem

- **Angular**: Provides a complete ecosystem with a wide range of tools and features out-of-the-box, such as routing, form handling, HTTP client, and testing utilities.
- **React**: More flexible and modular, relies on third-party libraries for routing (React Router), state management (Redux), and other functionalities.

7. Component Structure

- **Angular**: Components are organized within modules, which provide a clear and structured way to manage an application's components, directives, services, and pipes.
- **React**: Components are often more loosely organized, and the structure can vary significantly depending on the developer's preference or the project's requirements.

8. Community and Support

- **Angular**: Strong corporate backing from Google and a large, active community. Regular updates and long-term support (LTS) for major versions.

- **React**: Backed by Facebook and supported by a vast community. Frequent updates and extensive resources for learning and troubleshooting.

Summary

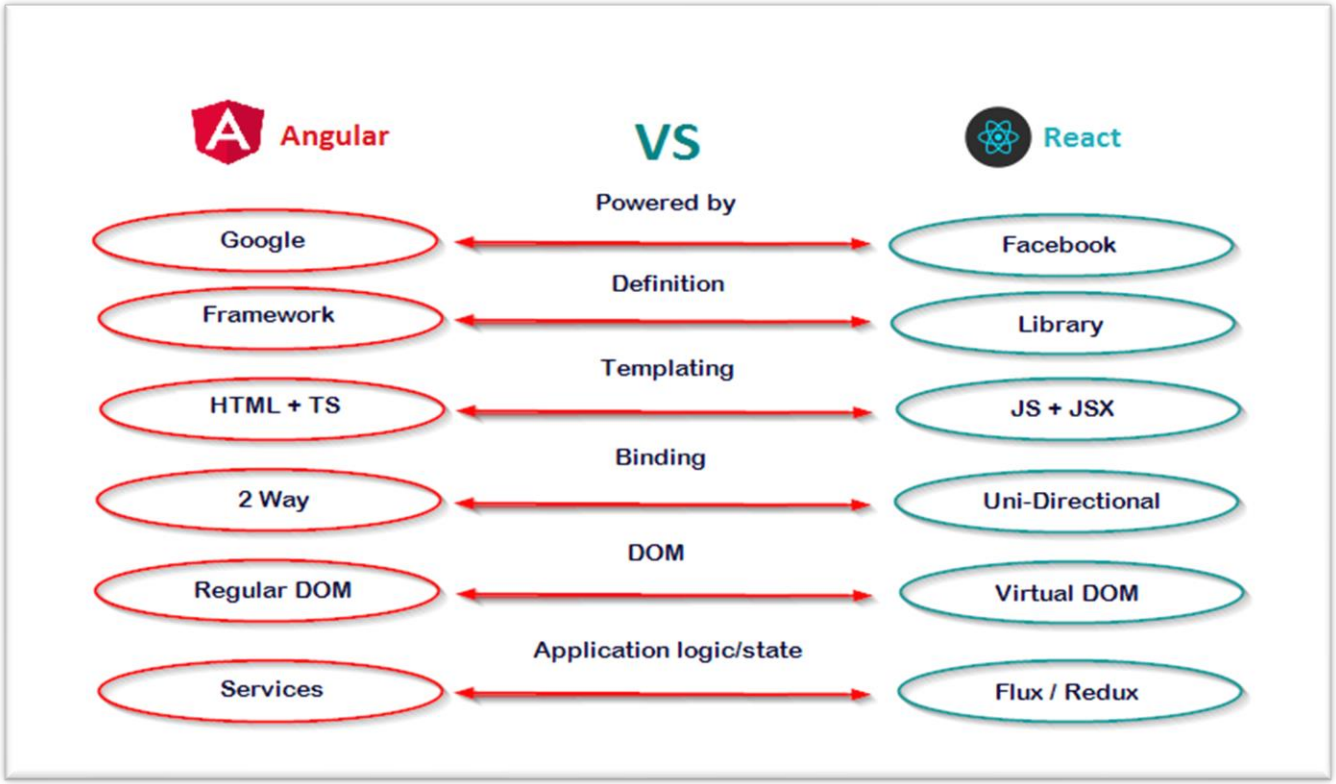
When to Choose Angular:

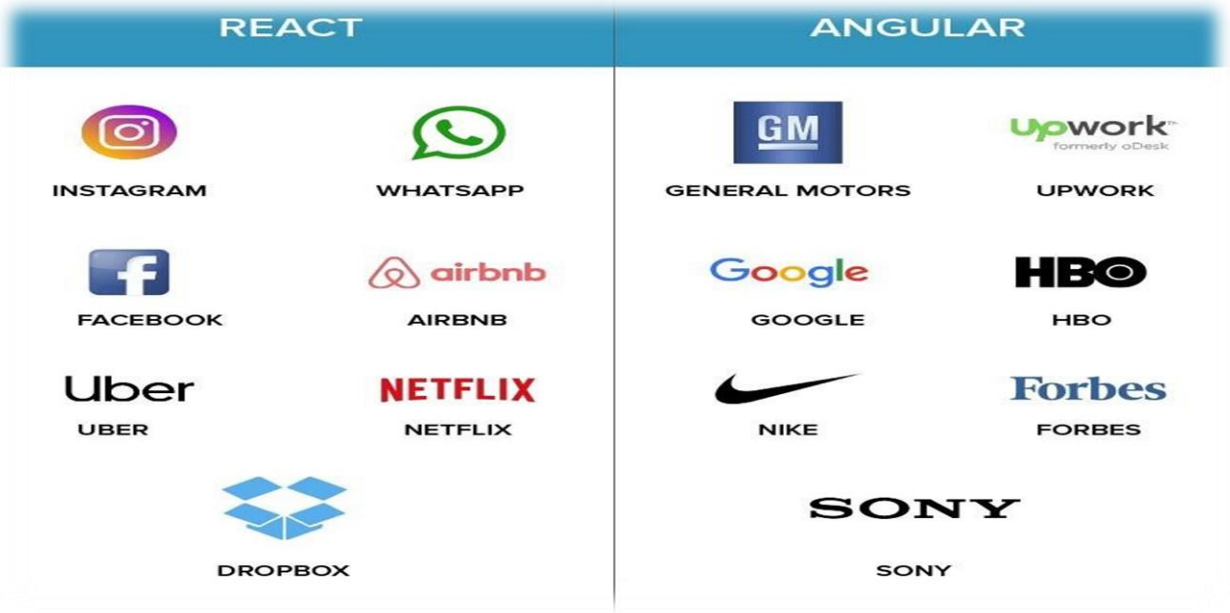
- You need a comprehensive framework that includes everything from routing to form validation out-of-the-box.
- You prefer TypeScript for its type safety and tooling benefits.
- You require robust support for large-scale enterprise applications.
- You favor two-way data binding for simpler synchronization between model and view.

When to Choose React:

- You need a flexible and lightweight library focused on the view layer.
- You are already familiar with JavaScript and prefer a gradual learning curve.
- You want to have more control over your application's architecture and prefer to integrate third party libraries for additional functionalities.
- You require high performance with efficient DOM manipulations.

Both Angular and React have their unique strengths and are well-suited for different types of projects. The choice between them often depends on the specific requirements of the project, the team's expertise, and the development approach preferred by the organization.


Angular
VS
React
Powered by
Google
Facebook
Definition
Framework
Library
Templating
HTML + TS
JS + JSX
Binding
2 Way
Uni-Directional
DOM
Regular DOM
Virtual DOM
Application logic/state
Services
Flux / Redux


REACT
ANGULAR
INSTAGRAM
WHATSAPP
GENERAL MOTORS
UPWORK
FACEBOOK
AIRBNB
GOOGLE
HBO
UBER
NETFLIX
NIKE
FORBES
DROPBOX
SONY
SONY

React JS

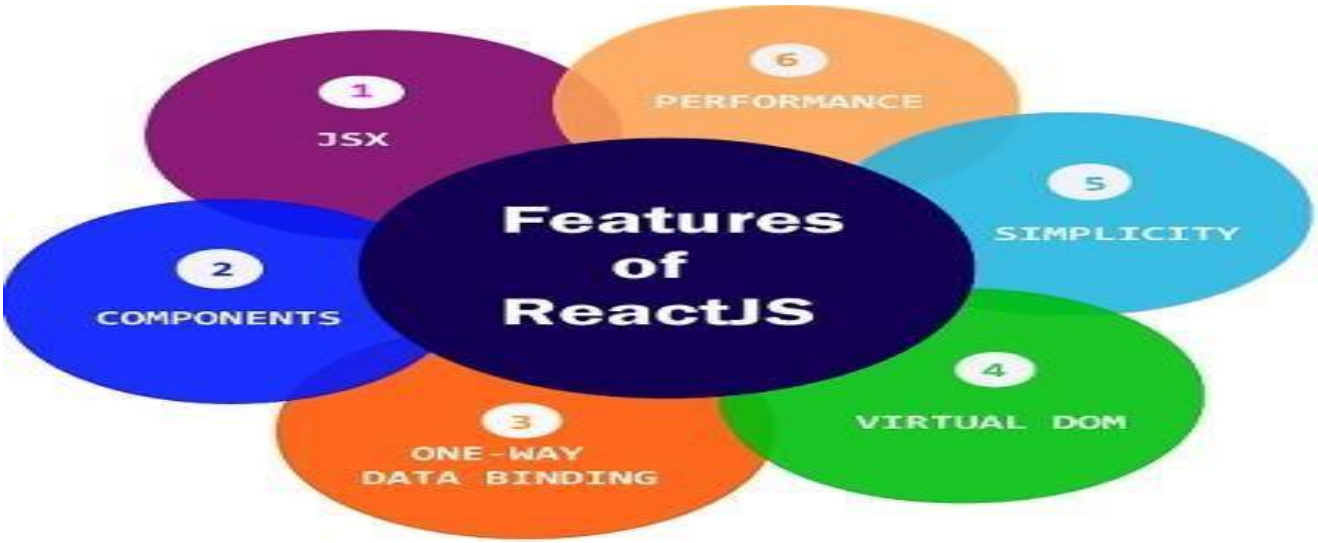
Unit 1: Introduction to React

What is React?

- **React**: A JavaScript library for building user interfaces, developed by Facebook.
- **Purpose**: To create single-page applications (SPAs) with a component-based architecture that allows for reusable UI components.
- **Initial Release**: 2013

Key Features

- **Component-Based Architecture**: Build encapsulated components that manage their own state and compose them to make complex UIs.
- **Virtual DOM**: Efficiently updates and renders components by using a virtual representation of the DOM.
- **Declarative Syntax**: Describes what the UI should look like, and React updates the UI automatically when the underlying data changes.
- **Unidirectional Data Flow**: Data flows from parent to child components, making the application more predictable.



Unit 2: Setting Up the Development Environment

Prerequisites

- **Node.js and npm**:

  Ensure Node.js and npm (Node Package Manager) are installed on your system.

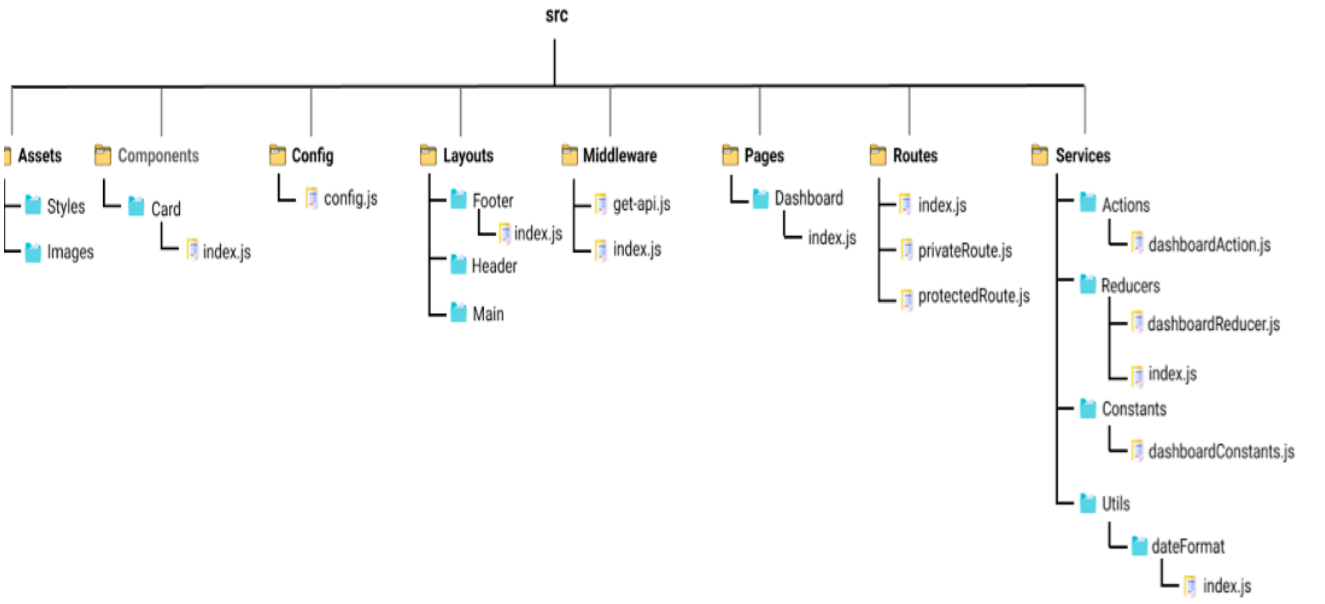
Creating a New React Project

- **Using Create React App**: A tool to set up a new React project with a sensible default configuration.

```
npx create-react-app myapp
```

```
cd my-app npm start
```

Folder Structure of React Application



```
my-react-app/
|
├── public/              # Public assets and the main HTML file
|   ├── favicon.ico      # Favicon for the application
|   ├── index.html       # Main HTML file
|   └── manifest.json    # Web app manifest
|
├── src/                 # Source files for the application
|   ├── assets/          # Static assets (e.g., images, fonts)
|   ├── components/      # Reusable components
|   |   ├── Header.js    # Example component
|   |   ├── Footer.js    # Example component
|   |   └── ...          # Additional components
|   |
|   ├── pages/           # Page components or views
|   |   ├── HomePage.js  # Example page component
|   |   ├── AboutPage.js # Example page component
|   |   └── ...          # Additional page components
|   |
|   ├── hooks/           # Custom hooks
|   |   ├── useAuth.js   # Example custom hook
|   |   └── ...          # Additional custom hooks
|   |
|   ├── services/        # API calls and business logic
|   |   ├── apiService.js # Example API service
|   |   └── ...          # Additional services
|   |
|   ├── utils/           # Utility functions and helpers
|   |   ├── formatDate.js # Example utility function
```

```
|   |   └── ...              # Additional utility functions
|   |
|   ├── App.js               # Main application component
|   ├── index.js             # Entry point for the application
|   ├── setupTests.js        # Test setup file
|   └── styles/              # Global styles and CSS files
|       ├── index.css        # Global CSS
|       └── ...              # Additional stylesheets
|
├── .gitignore               # Specifies which files and directories to ignore in Git
├── package.json             # Project dependencies and scripts
├── package-lock.json        # Exact version of dependencies installed
├── README.md                # Project overview and setup instructions
├── .eslintrc.json           # ESLint configuration
├── .prettierrc.json         # Prettier configuration
└── jsconfig.json            # JavaScript configuration (for IDEs)
```

Unit 3: React Components and JSX

Components

- **Functional Components**: Simplest form of components that are JavaScript functions.

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>; }
```

- **Class Components**: More complex components that extend React.Component and have lifecycle methods.

```
class Welcome extends React.Component {
render() {
```

```
    return <h1>Hello, {this.props.name}</h1>;
  }
}
```

JSX (JavaScript XML)

- **Syntax**: A syntax extension for JavaScript that looks similar to HTML and is used to describe what the UI should look like.

```
const element = <h1>Hello, world!</h1>;
```

Props

- **Purpose**: Pass data from parent components to child components. function Greeting(props) {
  return <h1>Hello, {props.name}</h1>;
  }

```
<Greeting name="Alice" />
```

Unit 4: State Management

useState Hook

- **Purpose**: Manage state in functional components.

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);     return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>Click me</button>
```

```
    </div>
  );
}
```

Class Component State

- **Purpose**: Manage state in class components.

```
class Counter extends React.Component {
constructor(props) {     super(props);
   this.state = { count: 0 };
  }    render() {
return (
     <div>
       <p>You clicked {this.state.count} times</p>
       <button onClick={() => this.setState({
       count: this.state.count  + 1 })}>
        Click me</button>
     </div>
   );
  }
}
```

Unit 5: Lifecycle Methods

Functional Components

- **useEffect Hook**: Performs side effects in functional components.

```
import React, { useEffect, useState } from 'react';
```

```
function Example() {    const [count, setCount] =
useState(0);

 useEffect(() => {
  document.title = `You clicked ${count} times`;
 });    return (
  <div>
   <p>You clicked {count} times</p>
   <button onClick={() => setCount(count + 1)}>Click me</button>
  </div>
 );
}
```

Class Components

- **Lifecycle Methods**:
  - **componentDidMount**(): Called after the component is mounted.
  - **componentDidUpdate**(prevProps, prevState): Called after the component updates.
  - **componentWillUnmount**(): Called before the component is unmounted and destroyed.

```
class Example extends React.Component {
componentDidMount() {
   document.title = `You clicked ${this.state.count} times`;
 }    componentDidUpdate(prevProps, prevState) {      document.title =
`You clicked ${this.state.count} times`;
 }   componentWillUnmount() {
  // Cleanup code
 }
}
```

Unit 6: Handling Events

Event Handling

- **Handling Events in React**: Similar to handling events in HTML but with JSX syntax.

```
function handleClick() {
alert('Button clicked');
}
function MyButton() {    return <button onClick={handleClick}>Click
me</button>; }
```

Unit 7: Conditional Rendering

Conditional Rendering Techniques

- **Using If Statements**: Inline conditional rendering.

```
function Welcome(props) {
if (props.isLoggedIn) {
return <UserGreeting />;
  } else {
    return <GuestGreeting />;
  }
}
```

- **Element Variables**: Store elements in variables and conditionally render them.

```
function LoginControl(props) {
const isLoggedIn = props.isLoggedIn;
  const button = isLoggedIn ? <LogoutButton /> : <LoginButton />;     return (
<div>
     {button}
   </div>
```

```
  );
}
```

Unit 8: Lists and Keys

Rendering Lists

- **Using map()**: Render a list of elements from an array.

```
function NumberList(props) {   const numbers =
props.numbers;    const listItems =
numbers.map((number) =>
   <li key={number.toString()}>
     {number}
   </li>    );
return (
<ul>
     {listItems}
   </ul>
  );

}
```

Unit 9: Forms

Controlled Components

- **Form Elements**: Manage form data with state.

```
import React, { useState } from 'react';

function NameForm() {
  // State to hold the name input value
```

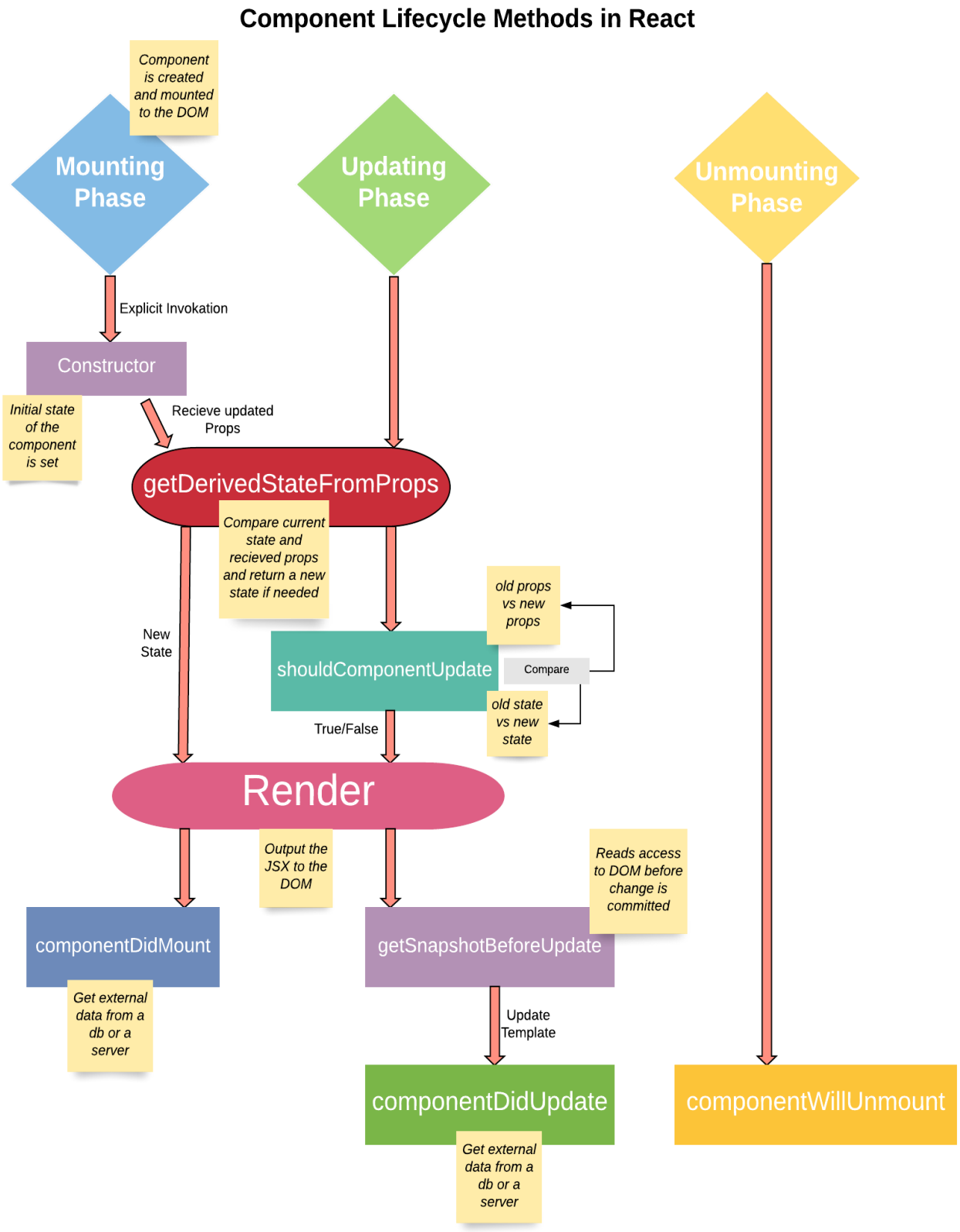
```
  const [name, setName] = useState('');

  // Function to handle form submission
  function handleSubmit(event) {
    event.preventDefault();
// Prevent the default form submission behavior
    alert('A name was submitted: ' + name);
// Show an alert with the name
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Name:
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
// Update state on input change
        />
      </label>
      <button type="submit">Submit</button>
    </form>
  );
}

export default NameForm;
```

Unit 10: Context API

Context API

- **Purpose**: Share state and methods between components without passing props explicitly.

```
const ThemeContext = React.createContext('light');   function
App() {
return (
   <ThemeContext.Provider value="dark">
     <Toolbar />
   </ThemeContext.Provider>
 );  }
function Toolbar() {
return (
   <div>
     <ThemedButton />
   </div>
 );  }
function ThemedButton() {
 const theme = React.useContext(ThemeContext);
return
<button
style={{ background: theme === 'dark' ? '#333' :  '#FFF' }}>
Theme Button
</button>;
}
```


Component Lifecycle Methods in React
Component is created and mounted to the DOM
Mounting Phase
Updating Phase
Unmounting Phase
Explicit Invokation
Constructor
Initial state of the component is set
Recieve updated Props
getDerivedStateFromProps
Compare current state and recieved props and return a new state if needed
New State
shouldComponentUpdate
old props vs new props
Compare
old state vs new state
True/False
Render
Output the JSX to the DOM
componentDidMount
Get external data from a db or a server
getSnapshotBeforeUpdate
Reads access to DOM before change is committed
Update Template
componentDidUpdate
Get external data from a db or a server
componentWillUnmount

Unit 11: Routing

React Router

- **Purpose**: Manage navigation and routing in a React application.

  Installation:

  npm install react-router-dom

- **Basic Usage**:

```
import { BrowserRouter as Router, Route, Switch } from 'reactrouterdom';  function
App() {
return (
<Router>
    <Switch>
      <Route path="/about">
        <About />
      </Route>
      <Route path="/">
        <Home />
      </Route>
    </Switch>
  </Router>
 );
}
```

Unit 11: Hooks

Custom Hooks

- **Purpose**: Reuse stateful logic between components.

```
import { useState, useEffect } from 'react';
```

```
function useCustomHook() {
const [value, setValue] =
useState(0);

  useEffect(() => {
    // side effect code
  }, []);
  return [value, setValue];
}
```

Unit 13: Performance Optimization

React.memo

- **Purpose**: Optimize functional components by memoizing them. const MyComponent = React.memo(function MyComponent(props) {
  // render component });

useMemo & useCallback

- **Purpose**: Optimize functions and values that depend on specific props or state.

  const memoizedCallback = useCallback(() => {
  // callback function
  }, [dependencies]);

  const memoizedValue = useMemo(() =>
  computeExpensiveValue(a, b), [a, b]);

Unit 14: Testing

Testing Library

- **Popular Libraries**: Jest (for testing), React Testing Library (for rendering components and querying DOM).
- **Basic Test**:

```
import { render, screen } from '@testing-library/react';
import App from './App';
test('renders learn react link', () =>
{
render(<App />);
const linkElement = screen.getByText(/learn react/i);
expect(linkElement).toBeInTheDocument();
});
```

---***---

Introduction to Vue.js

➢ What is Vue.js?

Vue.js is a progressive JavaScript framework used for building user interfaces. It focuses on the view layer, making it easy to integrate with other libraries or existing projects. Vue also provides a comprehensive ecosystem for building single-page applications (SPAs) when combined with modern tooling and supporting libraries.

➢ Overview of Vue.js

Vue.js is designed to be incrementally adoptable. This means you can start using it for a single feature or page and scale it up to a full-fledged application as needed.

Vue offers features like:

- **Reactive Data Binding**: Automatically syncs the view with the underlying data.
- **Component-Based Architecture**: Encapsulates code into reusable components.
- **Directives**: Special tokens in the markup that bind data to the DOM.

➢ Benefits and Use Cases

- **Ease of Learning**: Vue's syntax is straightforward, making it accessible for beginners.
- **Flexibility**: Suitable for both small and large projects.
- **Performance**: Efficient updates and rendering through a virtual DOM.

- **Community and Ecosystem**: Strong community support and a rich ecosystem of tools and libraries.

➢ **Use Cases**:

- **Single Page Applications**: Dynamic content without full page reloads.
- **Interactive Web Interfaces**: Forms, dynamic lists, and data visualizations.
- **Component Libraries**: Building reusable UI components.

❖ Comparisons with Other Frameworks

- **React**:
  - **React** is more focused on the view layer and requires a state management library and router separately, whereas Vue includes these as part of its ecosystem.
  - **React** uses JSX for templating, which integrates JavaScript logic and HTML, while **Vue** uses a template syntax that separates concerns.
- **Angular**:
  - **Angular** is a full-featured framework with a steep learning curve, whereas **Vue** is often praised for its simplicity and ease of integration.
  - **Angular** relies heavily on TypeScript, while **Vue** can be used with both JavaScript and TypeScript.

	Angular	React	Vue
Framework size	143k	97.5k	58.8k
Programming Lang	Typescript	Javascript	Javascript
Ui component	In-built material techstack	React UI tools	Component libraries
Architecture	component-based	component-based	component-based
Learning curve	steep	moderate	moderate
Syntax	Real DOM	Virtual DOM	Virtual DOM
Scalability	modular development structure	component-based approach	template-based syntax
Migrations	API upgrade	React codemod script	Migration helper tool

Angular Minds

2. Setting Up the Development Environment

Installation

- **Using Vue CLI**
  - **Install Node.js and npm**: Download and install from nodejs.org.
  - **Install Vue CLI**:

    npm install -g @vue/cli

  - **Create a New Project**:

    vue create my-project

Follow the prompts to select default configurations or customize settings.

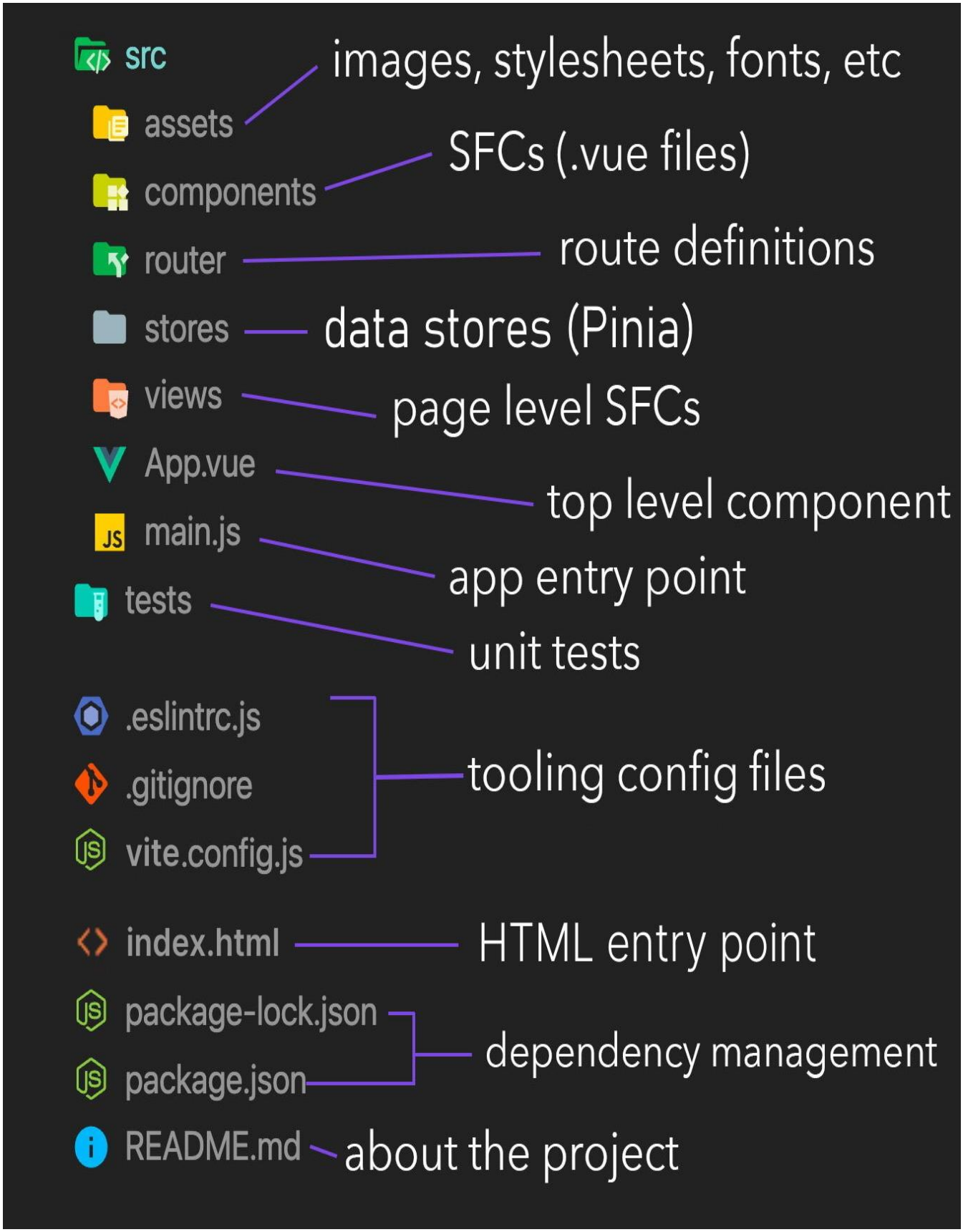
- **Run the Project**:

  cd my-project

  npm run serve

  The development server will start, and the application will be accessible at http://localhost:8080.

	ANGULAR	REACT	VUE
AUTHOR	Miško Hevery	Jordan Walke	Evan You
INITIAL RELEASE	May 2016	March 2013	February 2014
WRITTEN IN	TypeScript	JavaScript	JavaScript
ENTRY THRESHOLD	high	average	low
PROJECT SIZE	very complex	complex	smaller


src
images, stylesheets, fonts, etc
assets
SFCs (.vue files)
components
route definitions
router
data stores (Pinia)
stores
page level SFCs
views
App.vue
top level component
main.js
app entry point
tests
unit tests
.eslintrc.js
.gitignore
vite.config.js
tooling config files
index.html
HTML entry point
package-lock.json
package.json
dependency management
README.md
about the project

- **Using Vite**
  - **Install Vite**:

    npm create vite@latest my-project --template vue

  - **Install Dependencies**:

    cd my-project
    npm install

  - **Run the Project**:

    npm run dev

    The development server will start, and the application will be accessible at http://localhost:5173.

IDE Setup

- **Recommended Editors**:
  - **VSCode**: Lightweight, customizable with a vast extension ecosystem.
  - **WebStorm**: Comprehensive IDE with built-in support for Vue.js and other frameworks.
- **Useful Extensions**:
  - **Vetur**: For VSCode, provides syntax highlighting, IntelliSense, and more for Vue files.
  - **Volar**: For Vue 3, provides TypeScript support and Vue-specific tooling.

3. Vue.js Basics

Vue Instance

- **Creating a Vue Instance**:

```
new Vue({
  el: '#app',
  data: {
```

```
  message: 'Hello Vue!'
 }
});
```

This initializes a new Vue instance, binds it to an HTML element with the id app, and sets the data property message.

- **Data Binding**:

```
<div id="app">
 <p>{{ message }}</p>
</div>
```

Vue automatically binds the message data property to the DOM.

Templates and Directives

- **Basic Syntax**:

Vue templates are HTML-based and include Vue-specific syntax:

```
<div id="app">
 <p v-if="seen">Now you see me</p>
</div>
```

- **Directives**:
  - v-bind: Dynamically bind attributes:

    ```
    <img v-bind:src="imageSrc">
    ```

  - v-model: Two-way data binding:

    ```
    <input v-model="inputValue">
    ```

  - v-for: Render a list of items:

```
<ul>
  <li v-for="item in items" :key="item.id">{{ item.text }}</li>
</ul>
```

- v-if, v-show: Conditional rendering:

```
<p v-if="isVisible">Visible</p>
```

Event Handling

- **`v-on` Directive**:
  - Bind events to methods:

```
<button v-on:click="doSomething">Click me</button>
```

  - **Event Modifiers**:
    - `.prevent`: Prevent default behavior:

```
<form v-on:submit.prevent="submitForm">
  <!-- form elements -->
</form>
```

4. Vue Components

Component Basics

- **Single File Components (SFCs)**:
  - **Structure**:

```
<template>
  <div>
    <p>{{ msg }}</p>
  </div>
</template>
```

```
<script>
export default {
  data() {
    return {
      msg: 'Hello World!'
    };
  }
};
</script>

<style scoped>
p {
  color: red;
}
</style>
```

- **Component Registration**:

```
import MyComponent from './components/MyComponent.vue';

new Vue({
  components: {
    MyComponent
  }
});
```

Props and Events

- **Passing Data with Props**:

```
<my-component :someProp="parentData"></my-component>
```

In MyComponent.vue:

```
props: ['someProp']
```

- **Emitting Events**:

```
<button @click="$emit('customEvent', data)">Click me</button>
```

Slots

- **Named Slots**:

```
<template>
  <slot name="header"></slot>
  <slot></slot>
</template>
```

- **Scoped Slots**:

```
<template v-slot:default="slotProps">
  <p>{{ slotProps.text }}</p>
</template>
```

5. Vue Router

Installation and Setup

- **Install Vue Router**:

```
npm install vue-router
```

- **Configure Routes**:

```
import { createRouter, createWebHistory } from 'vue-router';
import Home from './components/Home.vue';
import About from './components/About.vue';
```

```
const routes = [
  { path: '/', component: Home },
  { path: '/about', component: About }
];

const router = createRouter({
  history: createWebHistory(),
  routes
});

export default router;
```

Dynamic Routing

- **Route Parameters**:

```
{ path: '/user/:id', component: User }
```

- **Query Parameters**:

```
this.$router.push({ path: '/user', query: { id: 123 } });
```

Navigation Guards

- **Global Guards**:

```
router.beforeEach((to, from, next) => {
  // Logic here
  next();
});
```

- **Per-Route Guards**:

```
const route = {
```

```
  path: '/profile',
  beforeEnter: (to, from, next) => { /* logic */ }
};
```

- **In-Component Guards**:

```
export default {
  beforeRouteEnter (to, from, next) {
    // Logic
  }
};
```

6. Vuex (State Management)

Introduction to Vuex

- **Store Setup**:

```
import { createStore } from 'vuex';

export default createStore({
  state() {
    return {
      count: 0
    };
  },
  mutations: {
    increment(state) {
      state.count++;
    }
  }
});
```

State, Mutations, Actions, and Getters

- **State Management Concepts**:
  - **State**: Centralized data store.
  - **Mutations**: Synchronous methods to modify state.
  - **Actions**: Asynchronous methods that commit mutations.
  - **Getters**: Accessor methods for computed properties based on state.
- **Dispatching Actions and Committing Mutations**:

```
this.$store.commit('increment');
this.$store.dispatch('someAction');
```

Modules

- **Namespaced Modules**:

```
const moduleA = {
  namespaced: true,
  state() {
    return { /* state */ };
  },
  mutations: { /* mutations */ },
  actions: { /* actions */ },
  getters: { /* getters */ }
};
```

7. Vue Composition API

Introduction to Composition API

- **`setup()` Function**:

```
export default {
```

```
  setup() {
    const message = ref('Hello Vue 3');
    return { message };
  }
};
```

- **Reactive State with `ref` and `reactive`:**
  - **`ref`:**

    ```
    const count = ref(0);
    ```

  - **`reactive`:**

    ```
    const state = reactive({ count: 0 });
    ```

Lifecycle Hooks

- **Using Hooks:**

```
import { onMounted, onUnmounted } from 'vue';

export default {
  setup() {
    onMounted(() => {
      console.log('Component mounted');
    });
    onUnmounted(() => {
      console.log('Component unmounted');
    });
  }
};
```

Composables

- **Creating Reusable Logic**:

```
import { ref } from 'vue';

export function useCounter() {
  const count = ref(0);
  function increment() {
    count.value++;
  }
  return { count, increment };
}
```

8. Vue Directives and Filters

Custom Directives

- **Creating and Registering Custom Directives**:

```
Vue.directive('my-directive', {
  bind(el, binding, vnode) {
    el.style.color = binding.value;
  }
});
```

Filters

- **Global Filters**:

```
Vue.filter('capitalize', function(value) {
  if (!value) return '';
  return value.toString().charAt(0).toUpperCase() + value.slice(1);
});
```

- **Local Filters**:

```
export default {
  filters: {
    capitalize(value) {
      if (!value) return '';
      return value.toString().charAt(0).toUpperCase() +
value.slice(1);
    }
  }
};
```

9. Vue and HTTP Requests

Fetching Data

- **Using Axios**:

```
npm install axios
```

- **Making Requests**:

```
import axios from 'axios';

axios.get('https://api.example.com/data')
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.error(error);
  });
```

Error Handling

- **Managing Loading and Error States**:

```
data() {
  return {
    loading: false,
    error: null,
    data: null
  };
},
methods: {
  fetchData() {
    this.loading = true;
    axios.get('https://api.example.com/data')
      .then(response => {
        this.data = response.data;
      })
      .catch(error => {
        this.error = error;
      })
      .finally(() => {
        this.loading = false;
      });
  }
}
```

10. Vue CLI and Build Tools

Vue CLI Features

- **Project Configuration and Plugins**:
  - **vue.config.js**: Customize project settings like proxy configurations, build settings, etc.

Building for Production

- **Optimization Tips**:
  - **Code Splitting**: Dynamically load components as needed.
  - **Lazy Loading**: Load components only when they are needed.
  - **Minification**: Minify JavaScript and CSS files.
- **Deployment Strategies**:
  - **Static Hosting**: Use platforms like Netlify or Vercel for static deployments.
  - **Server-Side Deployment**: Use services like Heroku or AWS for full-stack applications.

11. Testing in Vue.js

Unit Testing

- **Using Vue Test Utils and Jest**:
  - **Setup**:

```
npm install --save-dev @vue/test-utils jest
```

  - **Writing Tests**:

```
import { mount } from '@vue/test-utils';
import MyComponent from '@/components/MyComponent.vue';

test('renders props.msg when passed', () => {
  const wrapper = mount(MyComponent, {
    props: { msg: 'Hello World' }
```

```
  });
  expect(wrapper.text()).toMatch('Hello World');
});
```

End-to-End Testing

- **Using Cypress**:
  - **Setup**:

    ```
    npm install --save-dev cypress
    ```

  - **Writing Tests**:

    ```
    describe('My Test Suite', () => {
      it('Visits the app', () => {
        cy.visit('http://localhost:8080');
        cy.contains('Hello Vue!');
      });
    });
    ```

12. Advanced Topics

Server-Side Rendering (SSR)

- **Introduction to Nuxt.js**: Nuxt.js is a framework based on Vue.js that supports server-side rendering and static site generation out of the box.
  - **Setup**:

    ```
    npx create-nuxt-app my-nuxt-app
    ```

  - **Features**:
    - Automatic routing
    - Server-side rendering
    - Static site generation

Static Site Generation (SSG)

- **Using Nuxt.js for SSG**: Nuxt.js can generate static HTML files that can be deployed to any static hosting service.

Performance Optimization

- **Lazy Loading**:

```
const Home = () => import('./components/Home.vue');
```

- **Code Splitting**: Vue Router supports dynamic imports to achieve code splitting.

Internationalization (i18n)

- **Using Vue I18n**:

```
npm install vue-i18n
```

- **Setup**:

```
import { createI18n } from 'vue-i18n';

const i18n = createI18n({
  locale: 'en',
  messages: {
    en: { welcome: 'Welcome' },
    fr: { welcome: 'Bienvenue' }
  }
});
export default i18n;
```